

2nd year internship report

presented by

Matteo PROVERBIO



Enterprise :
USC CiTIUS

Enterprise tutor :
Juan Antonio CORRALES RAMON

SIGMA Clermont tutor:
Chedli Belhassen BOUZGARROU

Date : September 2025

Référence du fichier : Proverbio_ Matteo_RapportStage2A.pdf

FICHE D'IDENTIFICATION DU DOCUMENT

Rapport de Stage 2^{ème} Année		
TITRE DU DOCUMENT : Rapport de stage 2 ^{ème} année <i>2nd year internship report</i>		
AUTEUR(S) : Matteo PROVERBIO –MRS – Pôle mécanique		
Date du document	Nbre de pages	Référence du document
12/09/2025	28	Proverbio_Matteo_RapportStage2A.pdf
ABSTRACT: Presentation of the second-year internship as a researcher at the technology research center affiliated with the University of Santiago de Compostela CiTIUS. Installation and handling of a new work environment on a Linux distribution in order to manipulate an ur30 robotic arm through SensONE T80 force sensors and motion planning within the ROBOTA-SUDOE project.		
Keywords: report, project, robotic arm, effort sensor, motion planning, ur30		
RESUME : Présentation du stage de deuxième année en tant que chercheur au centre de recherche technologique affiliée à l'université de Saint Jacques de Compostelle CiTIUS. Installation et prise en main d'un nouvel environnement de travail sur une distribution Linux afin de manipuler un bras robotique ur30 au travers de capteurs d'efforts SensONE T80 et de planification de mouvements dans le cadre du projet ROBOTA-SUDOE.		
Mots clés : rapport, projet, bras robotique, capteurs d'efforts, planification de mouvements, ur30		
Visa de l'entreprise : Ce rapport de stage a été visé par (Nom, Prénom, Qualité) : Date : Signature :		
Je demande que ce rapport soit confidentiel (non diffusé sur l'intranet de l'école) : ☐ OUI ☐ NON		

SUMMARY

SUMMARY 4

ACKNOWLEDGMENTS.....5

INTRODUCTION7

1.1. CiTIUS.....	7
1.2. The project	8
1.3. ROS.....	8
1.4. Moveit.....	9

THE WORKSPACE10

1.5. The robotic arm ur.....	10
1.5.1. UR30	10
1.5.2. UR5e.....	11
1.6. SensONE T80 from BOTA SYSTEMS.....	12
1.6.1. Serial.....	12
1.6.2. EtherCAT	12

THE KNIFE 14

1.7. 3D print	14
1.8. The handle.....	15
1.8.1. Arduino.....	17
1.8.2. Python.....	18
1.8.3. Neural network	19
1.9. Alternative with OnRobot HEX-E v2	20
1.10. Flexible handle.....	21

SIDE TASKS 22

1.11. UR30 adaptor.....	22
1.12. 3D printed serial sensor	23
1.13. 3D simulation of the handle.....	24
1.13.1. Sensors view	24
1.13.2. Manipulability ellipse	25
1.14. Virtual machine.....	26

CONCLUSION 28

ANNEXE 29

ACKNOWLEDGMENTS

I express my sincerest acknowledgements to Mr CORRALES RAMON for the opportunity of this internship and the support he provided during those 4 months.

I address my acknowledgements to Saltanat SEITZHAN for the help she gave us during our internship and during her PhD.

Finally, I thank warmly the whole CiTIUS team for their welcome and for providing us with all the help we needed.

INTRODUCTION

1.1. CiTIUS

My internship took place at CiTIUS (Centro Singular de Investigación en Tecnoloxías Intelixentes), located at the University of Santiago de Compostela in Galicia, Spain. CiTIUS is a leading interdisciplinary research center created in 2009 that focuses on the development and application of intelligent technologies about computer science, artificial intelligence, robotics, mathematics and engineering. For four months, I worked on a robotic project with Universal Robots.

Furthermore, CiTIUS is linked with the academical universe on one hand, and on the other hand with the industrial world. It has several academic partners such as the University of Santiago de Compostela, the university of Vigo and A coruña.

But it has also several industrial partners as we can see on Figure 1 : CiTIUS' partners



Figure 2 : Front of CiTIUS



Figure 1 : CiTIUS' partners

Inside, there are some places dedicated to several activities :

The Figure 4 shows the laboratory where the interns work. I passed the most of my time here.

The Figure 3 shows the Maker Lab, the place where are used the 3D printers and machine tool.

The Figure 5 shows the laboratory where are stored the microelectronics equipment, such as solder for instance.



Figure 4 : Laboratory



Figure 3 : Maker Lab



Figure 5 : Microelectronics lab

1.2. The project

This project, called ROBOTA-SUDOE, is divided into two different parts :

The first one is about the cut of meat.

The second one is about the assembly of the different parts of a doll.

All the previous tasks are realized with the help of a serial robotic arm UR30. During this internship, I was charged to program the trajectory of the arm and to adapt the different sensors in it.

At the end, the robotic arm will have to support a human user in the cut of meat by providing enough power to cut meat without any effort.

The ur30 will also be autonomous in the doll assembly and will reduce the assembling time thanks to computer vision, trajectory program and tactile sensors.

The link for the Github where is the entire content of this report is at Annexe A.

1.3. ROS

First, I had to learn to use ROS (Robot Operating System), a new environment made for this kind of missions, on a Linux distribution called Ubuntu 20.04.

Then, I became familiar with ROS and its different applications, such as Moveit, used for the arm movements planification.

[Ros Topic]

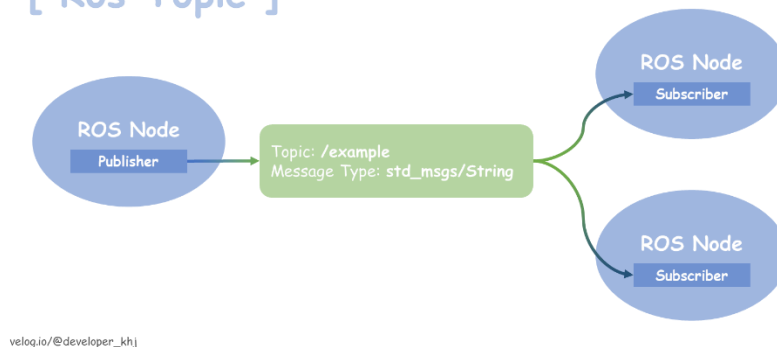
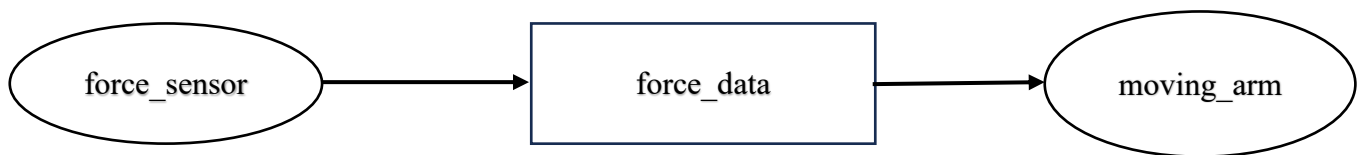


Figure 6 : ROS scheme

Here is how ROS is working. A program (in python or C++ language) is created. It can be a publisher or a subscriber, their role on the robotic moves depends on it. The publisher will publish some data to a topic. It can be everything : force, torque, temperature, acceleration... Then, another program (the subscriber) will get those data from this topic to use them (for moving the robotic arm for example). Those two programs are called nodes, and they can't communicate, they have to pass through a dedicated topic.

For instance :



In this case, a node called `force_sensor` get the data from the force sensor and publishes them to the topic called `force_data`. Then, another node called `moving_arm` gets the data from the topic `force_data` in order to move the robotic arm according to what the sensor gets.

1.4. Moveit

Moveit is one of the most widely used motion planning frameworks in ROS. It contains a powerful set of tools for developing applications involving robot motion planning, manipulation, 3D perceptin, kinematics, control and collision detection. At the beginning, we first used Moveit and Rviz to move the robotic arm through our computer. We moved the arm inside the simulation and the arm moved also in the reality. Then, we used Moveit to calculate the inverse of a Jacobian matrix to move the arm with inverse kinematics. We also implemented some volumes to create a security space where the arm can't go. Finally, we needed to be faster and to work with a high frequency, which were impossible to do with Moveit. That's why we found a way to make the calculus ourselves and to not use Moveit.

Furthermore, we also used Moveit to implement the gripper we will use in the doll assembly, to predict which move the arm will do and to avoid a collision between the gripper and an arm's joint.

THE WORKSPACE

1.5. The robotic arm ur

During my internship, I worked with two collaborative robotic arms from Universal Robots : the UR5e and the UR30. Both of them belong to the e-series family and are used in industrial and research environments due to their flexibility, safety features and ease to integration.

1.5.1. UR30



Figure 7 : UR30

The UR30 is a recent addition to the UR e-series lineup. It offers a higher payload capacity of 30kg while maintaining a reach of 1300mm with 6 degrees of freedom and a repeatability of 0.05mm more or less. It is more often used for heavy payload applications, and it will be used for the two aspect of the ROBOTA SUDOE project. But first, to make some test before using the UR30, we used a smaller Universal Robots : the UR5e.

1.5.2. UR5e



Figure 8 : UR5e

The UR5e is a versatile 6-axis collaborative robot with a payload capacity of 5kg and a reach of 850mm. It is well-suited for tasks such as pick-and-place or testing and has a repeatability of 0.03mm more or less. That's why we used it for testing our nodes and the security on our program.

However, during a test, the UR30 had a malfunction and we couldn't use it then. There was a security message about a mechanical break who broke, which was not the case, but the arm were in security mode and we couldn't use it anymore. That's why we used the UR5e to run all our tests after that.

1.6. SensONE T80 from BOTA SYSTEMS

The main goal of this project is to get the forces through sensors in order to help a human user to cut meat without any efforts. In that purpose, we are using two different captors SensONE T80 from BOTA SYSTEMS, one serial and the other etherCAT.

1.6.1. Serial

The serial sensor is the first sensor we succeeded to use. Actually, it is easier to use it than the etherCAT one. All we needed to do was to download the right files and packages from the official website and then, we were able to get the data, it means the 3 forces and the 3 torques through a USB link. But in order to create an enslavement of our robotic arm, it is not enough. We have to publish those data in a topic we called `/force_sensor` dedicated to this task : it's the publisher. Then, we created another code to get those data and to adapt the UR30 behavior : it's the subscriber who knows where to move the arm.



Figure 9 : Serial force sensor from BOTA SYSTEM

1.6.2. EtherCAT

The etherCAT sensor is more complex than the serial one. First, it is linked through ethernet, which enable to be faster (about $10\mu s$). To exchange data with the sensor, the Master (the computer) send a bus through the wire. Then, a slave (our sensor) fulfil the bus with its data (3 forces and 3 torques), and sends them back to the Master. Finally, the Master read the data show them to the user. To do that, the sensor must have the access to the network card, which is quite complicated to get because of the computer's safety.



Figure 10 : EtherCAT force sensor from BOTA SYSTEMS

However, we had an issue with those two sensors. At first, we used 3D printed supports on which were mounted the sensors. It worked but there were some slacks, that's why we asked to a partner company, CENTIMFE, to make us the same support with metal. We claimed them two months after we arrived. Then, we decided to mount the sensors on those new supports, but they didn't work anymore without knowing why. They sent None signal, so we unmounted them to try to get data, but the issue didn't change. We contacted the support, and there is no explanation for this behaviour. We are assuming that maybe, when we mounted the sensor on the metallic supports, there was some static electricity which broke the inside of the two sensors, which is quite strange because it is industrial sensors, with electric norms, so this would be quite astonishing.

To keep doing some test with the UR5e, we decided to use the integrated sensor of the Universal Robots:

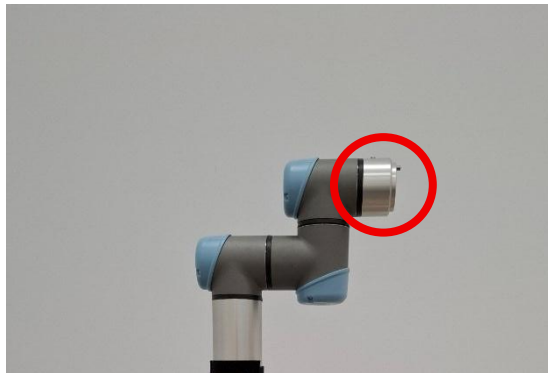


Figure 11 : Integrated sensor

The light grey part is an integrated sensor we can find in every Universal Robots. However, they are less accurate and it is harder to move the arm like that, that's why we were using two others sensors from BOTA SYSTEMS. Finally, we succeeded to use it and keep doing some experiments with the arm.

THE KNIFE

1.7. 3D print



Figure 12 : 3D printed knife

To realize some tests, we needed to use a fake knife for more safety. That's why we used 3D printed supports and 3D printed handle and blade. Later, we would need a more complex handle, but, at the beginning we used a simple and rigid handle to manipulate the robotic arm. Even after the issue with the metallic supports, we used this handle with this support. It was a lot easier to apply torques on the sensors then. We also decided to make the yellow support in one piece instead of three like it was the case before. Even if it is still plastic, it reduced the slacks.

Then, when we got the metallic ones from CENTIMFE, we just needed to print an adapted handle to mount it on the metallic support, since we had to change the shape of the handle to add a button and several sensors on it.

We also wanted to add a security button on the handle : when the user presses the button, the robotic arm turns on or turns off.

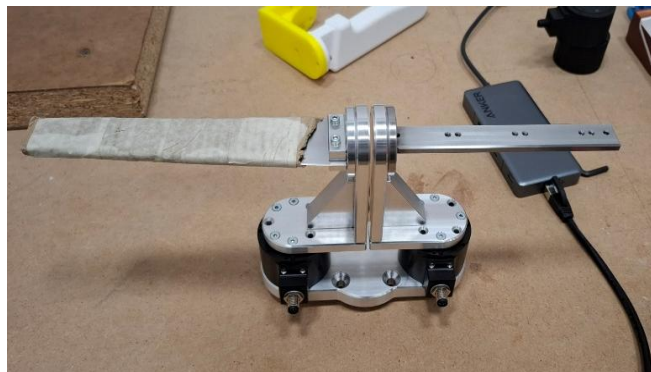


Figure 13 : Metallic supports from CENTIMFE

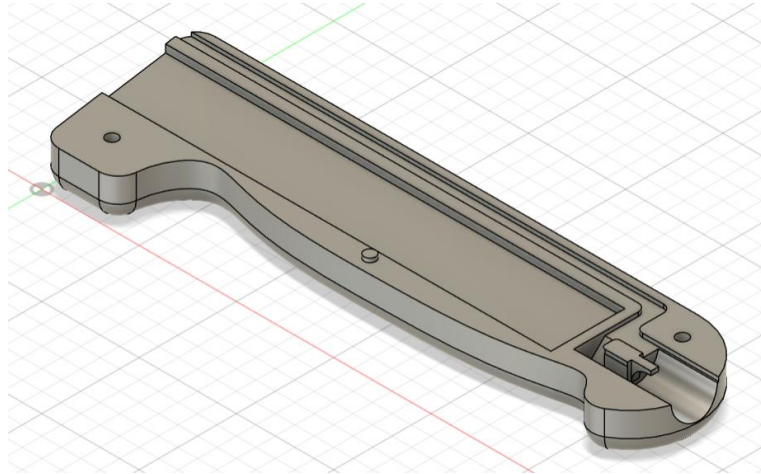


Figure 14 : Adapted handle for button and wires

1.8. The handle

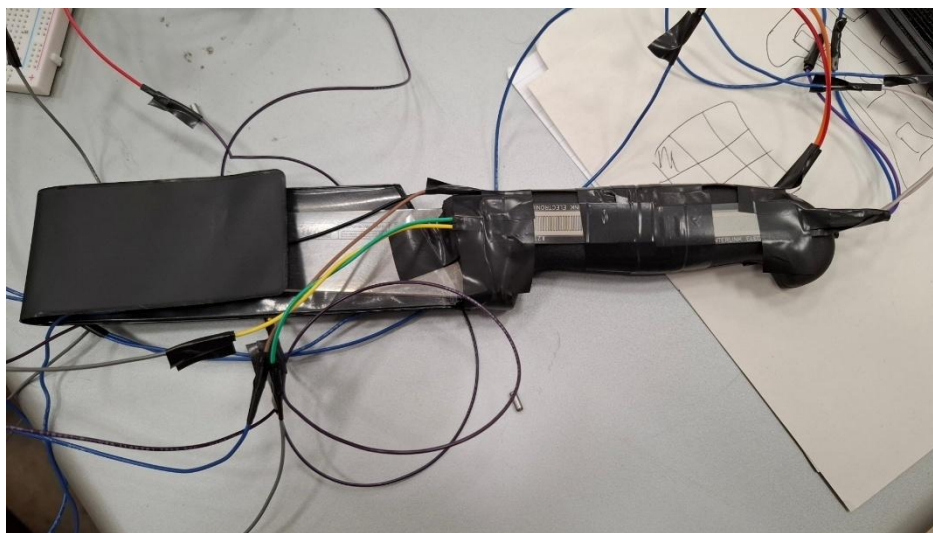


Figure 15 : The real knife

Once we succeeded to manipulate the robotic arm with the 3D printed knife, we were searching for a way to predict how we are holding the knife : in a standard way or in a reverse way. For that, we needed to put several tactile sensors on the handle. We put six with tape, two on a side, two on the other side and two above as we can see on the 0Figure 15. The purpose was to train a neural network to succeed to guess how we are holding it. At first, we created a csv file to study the data, and we noticed that on the six sensors, only two were required. But, to confirm that and to have more data, we kept the six sensors and we trained the neural network with them.

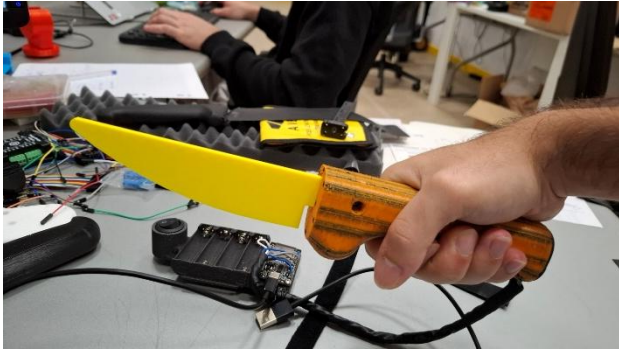


Figure 17 : Standard mode



Figure 16 : Reverse mode

However, there were some issues with the sensors we used on the handle fixed on the robotic arm : the wires were too much tough. That's why, when we were mounting them, some sensors 'pins broke as we can see on Figure 18.



Figure 18 : Sensors on the fixed handle

To fix that, we replaced the tough wires by flexible ones, so it was easier to move the sensors without damaging the pins. In addition to that, we oriented the wires in the same way (to the blade) to avoid to damage the pins.

1.8.1. Arduino

In order to get the data from the sensors, we used the Arduino IDE with a Teensy card. We put a resistor and a sensor in a series, and we made this six times in parallel (Figure 21). Then, we were able to get and see the data on the serial monitor (Figure 19).

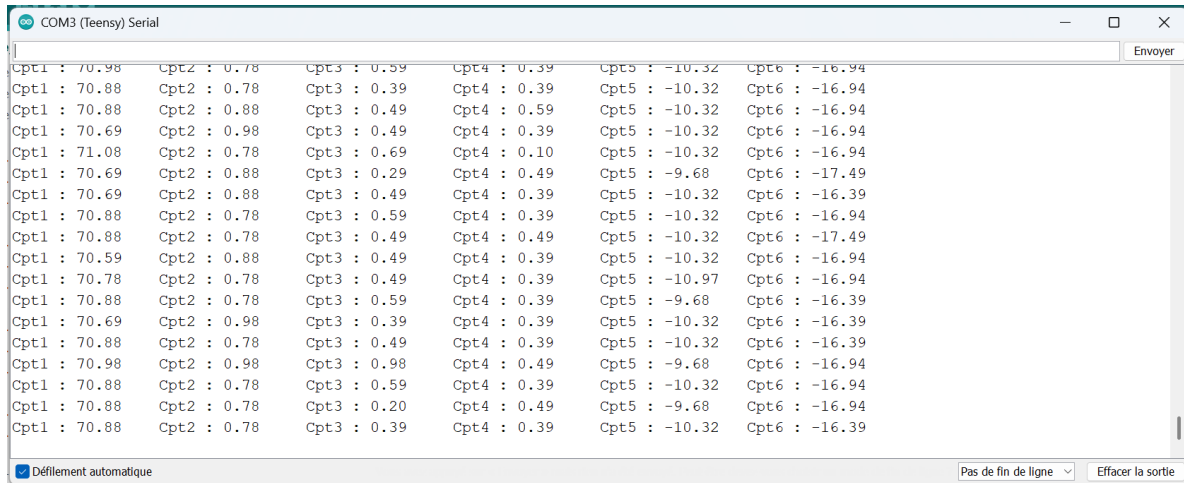


Figure 19 : Data from the sensors on the handle

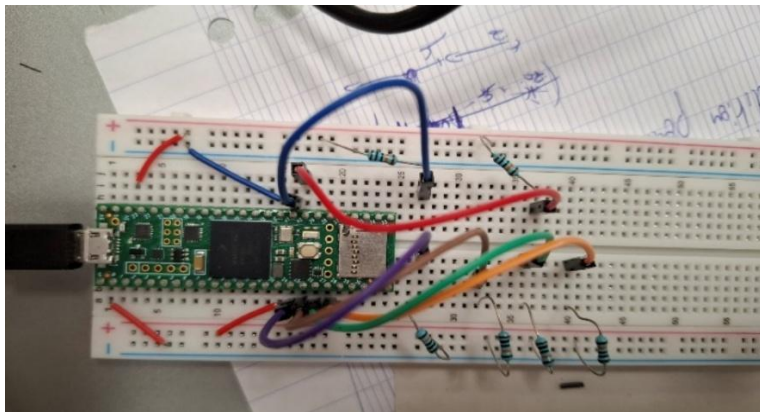


Figure 21 : Teensy set-up

First, we were using small sensors, but then we noticed that it didn't fit according to the person holding the knife. Some would touch the sensors, and others wouldn't. So, we decided to put longer sensors on it to become independent from the person holding the handle.



Figure 22 : Small tactile sensor

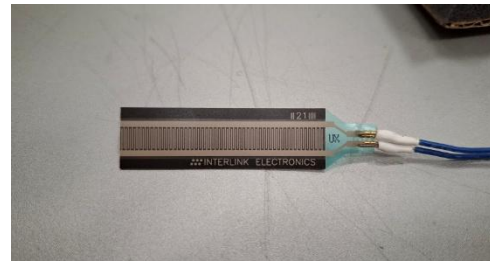


Figure 23 : Long tactile sensor

Then, we needed to make the data readable for the python code, that's why the data send by the Teensy card were in this form : value1,value2,value3,value4,value5,value6.

1.8.2. Python

At first, we tried to get the data on python to study them. In that purpose, we made two windows : one with the real time acquisition (Figure 24) to be sure that everything worked well, and a second one with a histogram where we could see the frequency of each value for each sensor every 5 seconds (Figure 25).

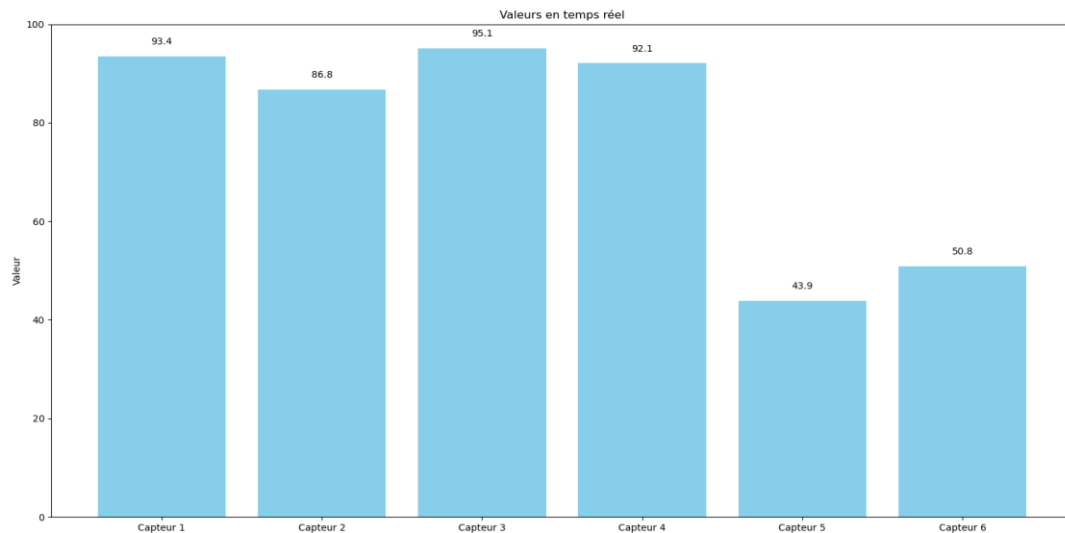


Figure 24 : Real time sensors

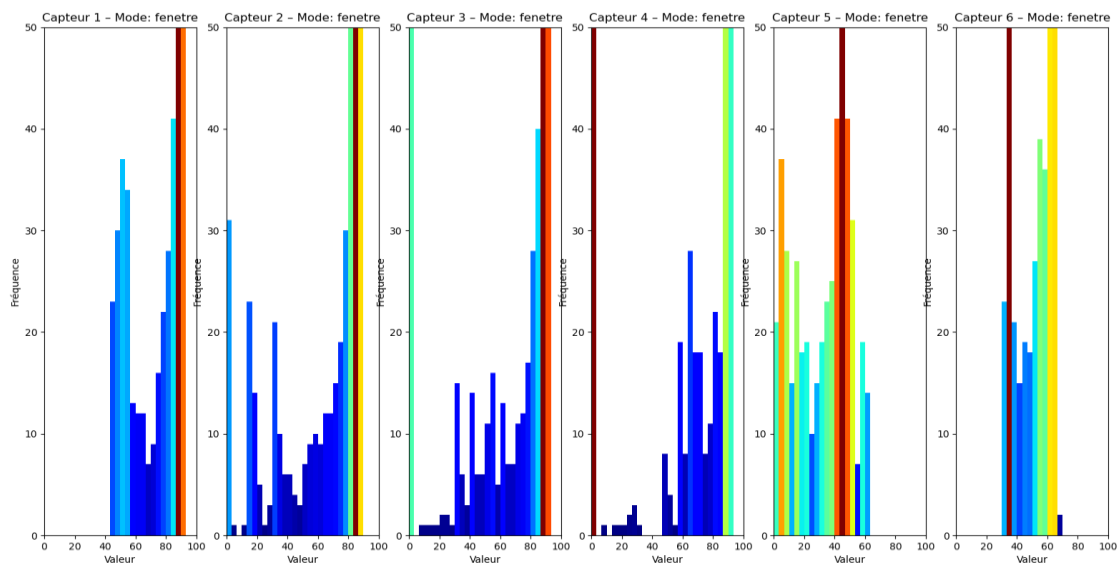


Figure 25 : Frequency of the sensors' figures

That's how we noticed that there was a problem : the sensors were disconnected very easily, we couldn't manipulate the handle properly. So, we soldered them.

The whole code is available at [Annexe A : Project GitHub link](#)
Protteo/ROBOTA_SUDOE

Annexe B. We have to take care of closing the serial port in the code before restarting it, otherwise Python couldn't read the serial port.

1.8.3. Neural network

In order to guess how we are holding the handle, we used a neural network to make the difference between three positions : when we are not holding the handle, when we're holding it in a standard way and when we're holding it in a reverse way.

The code provided in Annexe C creates a csv file in which there are the data for the three positions.

At the beginning, the program asks the user what he wants to do : the user can do an acquisition, a prediction or a training.

First, the user has to do an acquisition. The program asks the user if he wants to reinitialize the csv file (if it exists), then it asks how long the acquisition is for each position. Finally, the user can hold the handle through the three positions and complete the csv file automatically.

After the acquisition, the user has to train the neural network model.

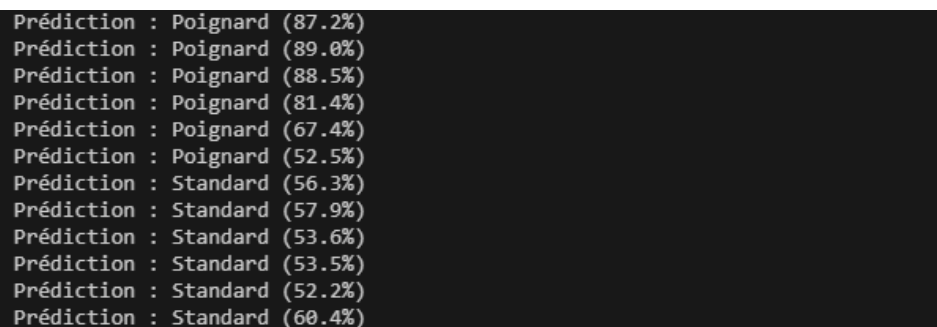
Once those two steps done, the neural network can start to predict the new positions. First, we had several bad guesses because the set-up with the sensors was not optimal. Then, we soldered the sensors and the data were cleaner.

But the purpose evolved. We didn't have to only guess in which position we are holding the handle, we also had to predict the movement right after. That's why we decided to put the handle in real conditions on the UR5e to try to guess the movement.

So, we first tried with a flexible handle described at 0. The purpose was to guess if we are holding the handle in standard mode or in reverse mode, with a straight trajectory or a curved trajectory to the right or to the left. But we didn't succeed to guess all the positions.

That's why we tried with the 3D printed handle described at 1.7 and the six tactile sensors. There, the results were better, but there were still some positions we didn't succeed to guess correctly. The code for this neural network is available at Annexe F.

Furthermore, since we were guessing a movement and not a position, we couldn't just look at the sensors' values one by one. That's why the neural network is trained on a window of several consecutive values for each sensors.



```
Prédiction : Poignard (87.2%)
Prédiction : Poignard (89.0%)
Prédiction : Poignard (88.5%)
Prédiction : Poignard (81.4%)
Prédiction : Poignard (67.4%)
Prédiction : Poignard (52.5%)
Prédiction : Standard (56.3%)
Prédiction : Standard (57.9%)
Prédiction : Standard (53.6%)
Prédiction : Standard (53.5%)
Prédiction : Standard (52.2%)
Prédiction : Standard (60.4%)
```

Figure 26 : Predictions of the neural network

Unfortunately, we didn't succeed to use it properly. When I trained the neural network, it guessed perfectly how I was holding the handle, but when another person tried, it was unable to give a correct prediction. I assume that maybe there were not enough sensors on the handle, or maybe we should train the neural network with the butcher who would use it.

1.9. Alternative with OnRobot HEX-E v2

Since the two Rokubi sensors broke, we didn't have any sensors able to get data from the handle and from the blade, which was the point to use two different sensors. So, to get those missing data, we just used a system to prove that the arm is efficient. For the handle part, we used the sensors described in 0. Furthermore, we needed to be sure that the transmitted force was the same in the handle and the blade.

For the blade part, we used another sensor : the OnRobot HEX-E v2, fixed on the table. However, to fix it we needed to print an adaptor since it was made for a Universal Robot effector.

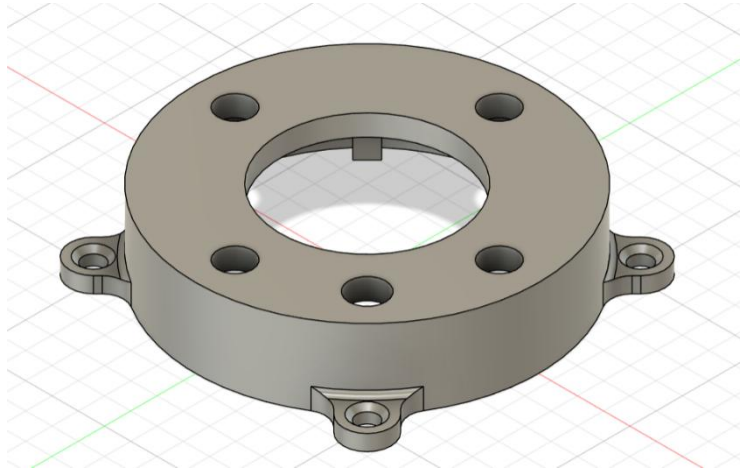


Figure 27 : Adaptor for the OnRobot sensor

I was surprised by the hole in the middle. I knew that the printer was not very accurate, so I designed it by adding 1mm to its diameter. But it didn't fit yet. That's how I learned that for common 3D printer, we need to add about 3mm in order to make fit holes correctly.



Figure 28 : Set-up to get the data from the blade

However, we encountered several issues with this sensor. There weren't official user manual to provide us the code and the driver to use this sensor, that's why we didn't have a good frequency by using it.

1.10. Flexible handle

To get data from the handle, we also used another kind of 3D printed material : a flexible plastic. Inside the handle, there are six sensors connected to a ESP32-WROOM-32D card. The code to see the result is available at Annexe D.

However, it is the first model and it is still a prototype, that's why we noticed several issues by using it. First, we couldn't identify which sensor is which. With the python program described at 1.8.2, we can see in real time the sensors' values, but we couldn't find which rectangle is associated to which sensor. Furthermore, two of the six sensors were sending the value 0, like they didn't work at all.

Finally, to try using this handle, we decided to test it with the neural network. Since we were not able to find which sensor was doing what, we thought that the neural network could make the difference to guess the way we were holding the handle. The code used is given at Annexe E. Unfortunately, even if the neural network could make the difference between the three positions, its accuracy was quite low. In fact, the way we are holding the handle in a classical way or in reverse way are very similar. That's why we thought to rise the size of the training data and to take different persons to take the handle. In that case, we could access to several ways to hold the knife for a same position in order to improve the accuracy of the neural network prediction.

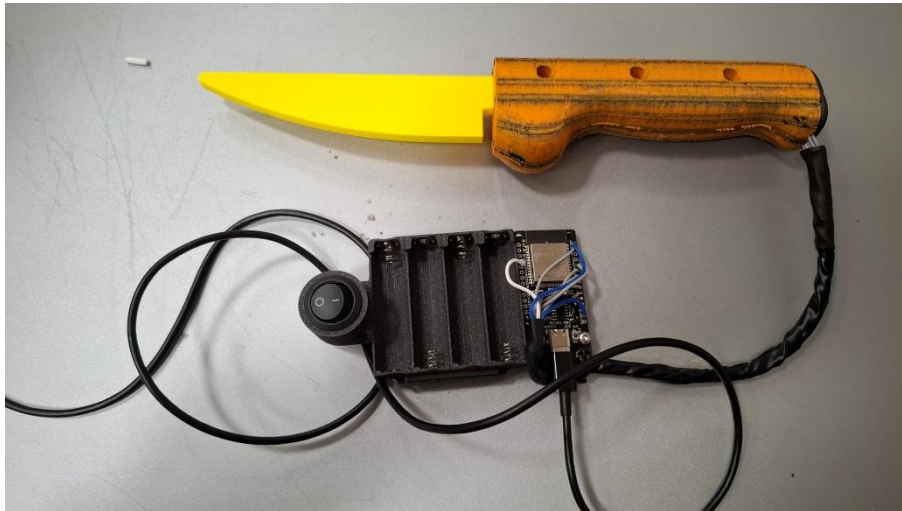


Figure 29 : Flexible handle with integrated sensors

At first, we used the neural network with this handle, but the low accuracy didn't enable us to guess correctly the way we are holding it.

SIDE TASKS

1.11. UR30 adaptor

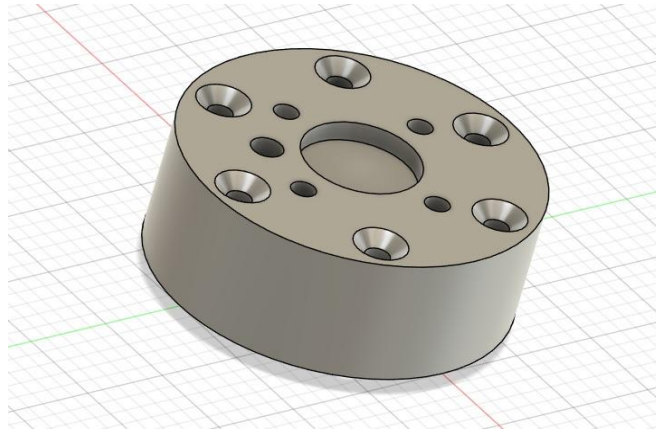


Figure 30 : UR30 adaptor

When we arrived, there was a sensor from BOTA SYSTEM mounted on the UR30, and on this sensor, there was the 3D printed supports of the knife. The purpose was to use it in addition to the integrated sensor and the two Rokubi sensors, to see which one is more efficient. However, the sensor has been broken. At first, we kept it to keep using the 3D printed support, but then we send it back to make it fix. So, I created this piece to mount the supports on the UR30 to proceed some tests on it instead of the UR5e.

The final mounting is shown in Figure 32.

1.12. 3D printed serial sensor



Figure 31 : 3D printed serial sensor

In order to realize some tests, we needed to manipulate the robotic arm UR5e. However, without the serial sensor mounted on it, we first used a plastic model sent by CENTIMFE to mount the blade on the metallic support. But quickly we found that the blade was not stable, so the data got weren't accurate. That's why we printed the same model as the serial sensor, but with placements we could use to screw with bolts (Figure 31).

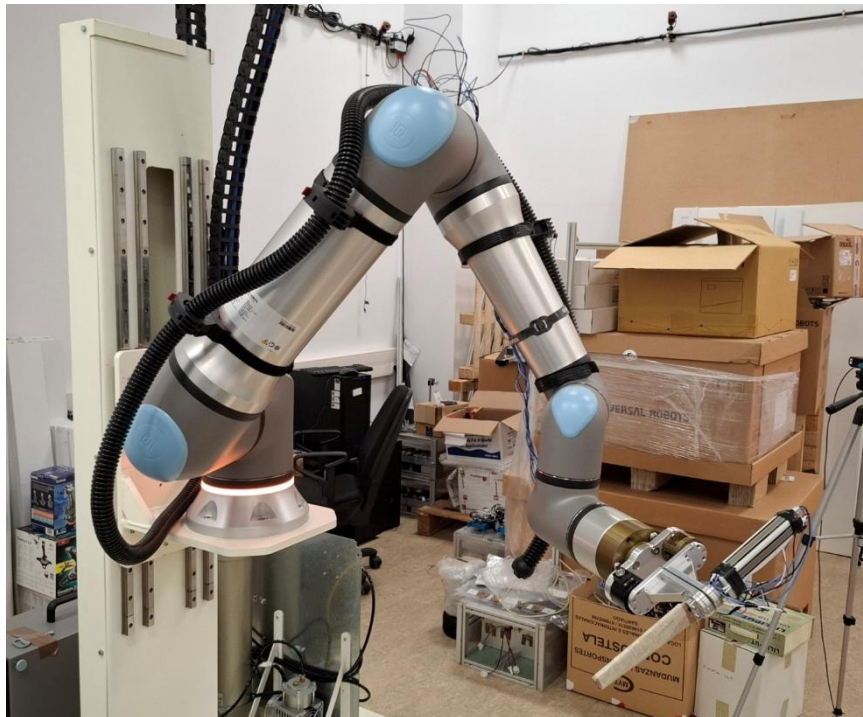


Figure 32 : Final mounting

1.13. 3D simulation of the handle

1.13.1. Sensors view

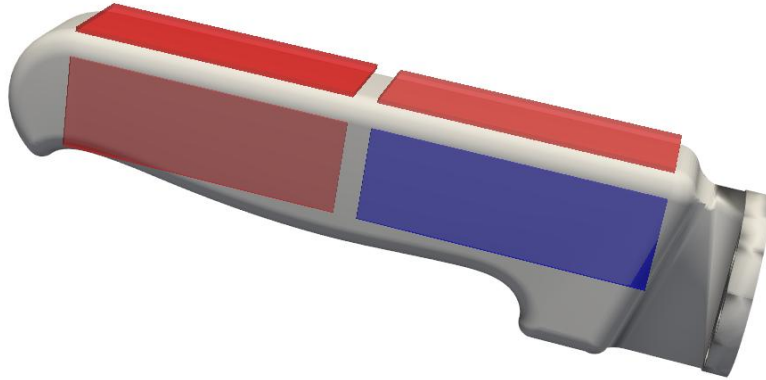


Figure 33 : 3D simulation with the handle and the sensors

With python, I was able to import the STL file of the handle to display it on a screen with the library **pyvista**. Above this model, I added several thin rectangles to imitate the six sensors. Then, I used it in several ways. First, I connected the rectangles with the real ones in real time through a serial port and the Arduino card connected to the computer. Then, when we manipulate the handle, we can see thanks to a colour gradient which sensor is the most solicited (dark blue when it is not and dark red when it is). The code is available at Annexe G.

Then, after doing several tests, we collected several data on csv files. I used those csv files to recreate the simulation with this model. So, if we want to replay the moment and just see the way sensor are solicited through the test, we can use this code available in Annexe H.

1.13.2. Manipulability ellipse

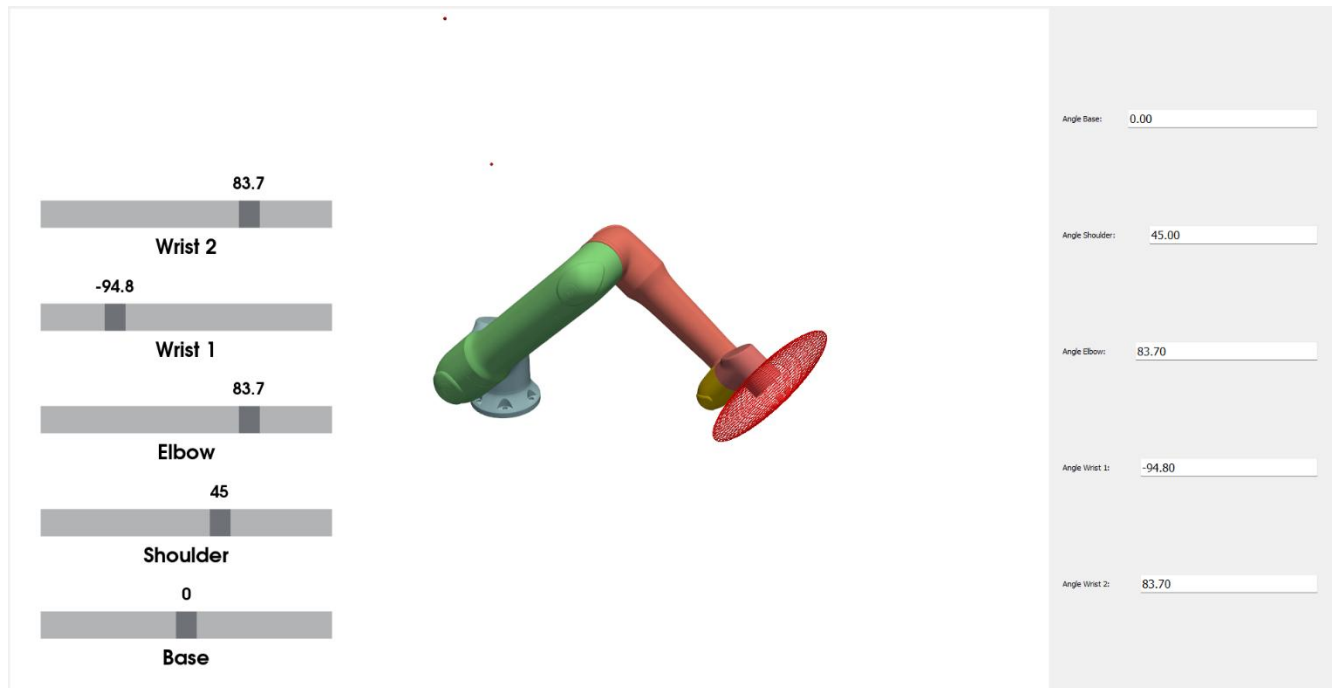


Figure 34 : Simulation of the robotic arm UR30 with its manipulability ellipse

Then, we thought it could be useful to have a visibility upon the manipulability ellipse of the robotic arm. This ellipse shows the arm's ability to move in the space according to the way it is. For example, if all the joints are straight, the ellipse will be flat because the arm could move in only one direction. It is useful to see the singularity for instance. First, I assembled the robotic arm in the graphic window. I downloaded the official fusion file from OnRobot website, then I isolated the different parts to turn them into a STL file. Then I placed them in the graphic window to display the UR30. To be able to move the arm, I associated each part to a cursor to move them. It is the direct kinematic. In order to be more precise, I also put an input window to set the joint angles as we want. After doing that, we needed to see the manipulability ellipse. I wrote the code at Annexe I to calculate the Jacobian matrix in order to know in which direction the robot can move, to display the ellipse. Then with the code provided at Annexe J, We can display the ellipse associated to the end effector as we can see in Figure 34.

1.14. Virtual machine

To conclude this internship, I wanted to deal more with Linux environment and ROS, that's why I gave myself another purpose : manipulate the ur5e with a joystick. Since my computer doesn't have ubuntu 20.04, I had to install a virtual machine with the app VirtualBox by Oracle. To connect the robot to the computer through a virtual machine, you have to make a bridge access and to choose the right name of the adapter you are using such as in Figure 35 in the network settings.

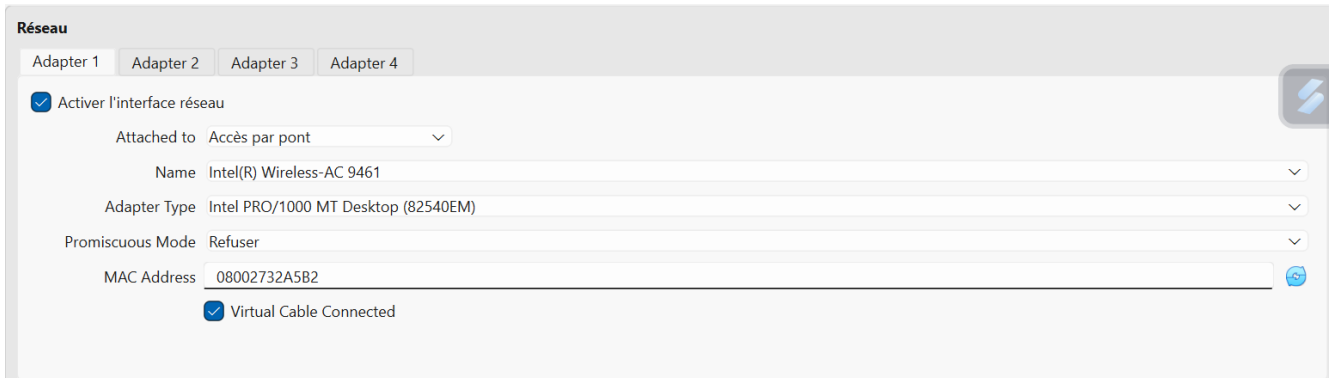


Figure 35 : Network set for virtual machine

Then, you have to set on the robot tab the IP address for an external controller and put the same IP address in your computer (Figure 38 and Figure 37) :

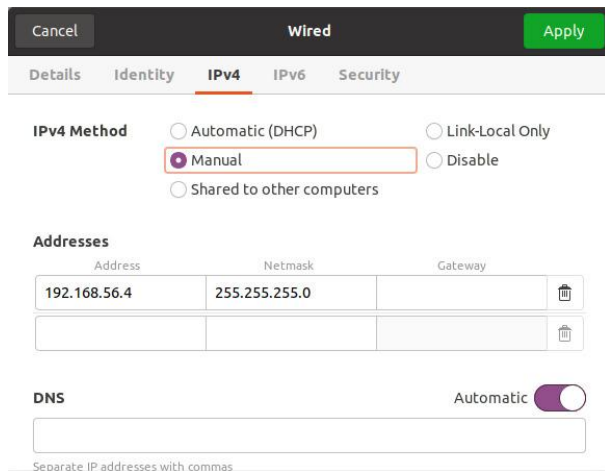


Figure 38 : Computer network settings



Figure 36 : Joystick with gazebo

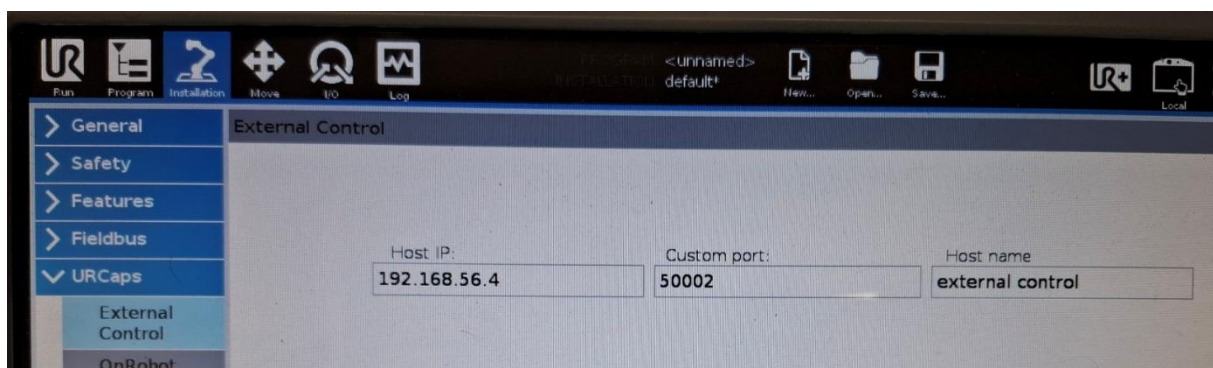


Figure 37 : ur5e external control settings

Unfortunately, I had to face an issue : I was not able to control the real ur5e because of the virtual machine. I used an adapter Ethernet/USB to connect my computer to the arm, and the link was established and verified with the command **ping 192.168.56.4**. However, the process was too slow and I was not able to establish a real time connection, I saw it when I launched the ur5e ROS driver. That's why I tested my codes with the ur5e simulation inside gazebo (Figure 36).

First, I used a code to move the arm through direct kinematic in gazebo (Annexe K). Then, I used a code to set a pose and an orientation through inverse kinematic (Annexe L) with MoveIt. After that, I wrote a publisher to map the joystick in order to control each joint through direct kinematic and without MoveIt (Annexe M) and to publish in the good topic. It worked with gazebo.

Every used commands to do this are listed in Annexe N.

Then I wanted to manipulate the arm with the joystick through inverse kinematic. Since I was able to publish the joints' velocities in the good topic, I had to do the same things with inverse kinematic. The formula I used is :

$$\begin{aligned}v_{effector} &= J * q \\ J^{-1} * v_{effector} &= q\end{aligned}$$

With :

$$\begin{aligned}v_{effector} &= \text{end effector velocity} \\ J &= \text{Jacobian matrix} \\ q &= \text{joints' velocities}\end{aligned}$$

Furthermore, with ROS, we can use KDL library to get the Jacobian matrix, it already knows the ur5e configurations.

First, I had to publish the joystick mapped data in the good topic with the right type (Annexe O). Then, the code available at Annexe P enable to get the Jacobian matrix and the joints' velocities for a desired velocity of the end effector (joystick data).

CONCLUSION

During this internship, I could learn about several engineering fields. I discovered ROS and how to use it in order to manipulate a robotic arm. I also could improve myself about 3D printing and about soldering. I could use Arduino with tactile sensors and translate the data with Python.

For my third year at SIGMA Clermont, I chose the mechatronics option, it means it is about mechanics, electronics, informatic and automatism. During those 4 months at CiTIUS, I could work about three on those four sectors, and it reaffirmed my choice to work in this area.

Furthermore, I could discover what research is and what we have to face during the process. Sometimes, events we don't expect happened and we had to find other solutions to fix the situation by thinking in a different way, what was very interesting, such as the HEX-E v2 described at 1.9.

Finally, working in ROBOTA-SUDOE project made me discover other problematics and topics to study, what was very interesting and enriching, and my experience at CiTIUS will be very useful as a future engineer.

ANNEXE

Annexe A : Project GitHub link
[Protteo/ROBOTA_SUDOE](#)

Annexe B : Python code to get data from the sensors

```
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import deque
import threading
import numpy as np
import time
from matplotlib.colors import LinearSegmentedColormap

from matplotlib.patches import Rectangle
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.widgets as widgets

# ----- CONFIGURATION -----
mode = "fenetre" # "continu" ou "fenetre"
interval_update_sec = 1 # fréquence de mise à jour histogramme (en secondes)
num_capteurs = 6
port_serial = "COM3"
baudrate = 9600
acquisition_active = True

# ----- INITIALISATION -----
ser = serial.Serial(port_serial, baudrate, timeout=1)
def on_close(event):
    global fenetres_ouvertes, acquisition_active, thread_acq
    fenetres_ouvertes -= 1
    if fenetres_ouvertes == 0:
        print("Toutes les fenêtres sont fermées. Fermeture du port série...")
        acquisition_active = False
        if thread_acq is not None:
            thread_acq.join(timeout=2) # Attend la fin du thread (max 2s)
        try:
            ser.close()
            print("Port série fermé proprement.")
        except:
            print("Erreur lors de la fermeture du port série.")

labels = [f"Capteur {i+1}" for i in range(num_capteurs)]
data_buffers = [deque() for _ in range(num_capteurs)] # Historique complet
data_fenetre = [deque() for _ in range(num_capteurs)] # Fenêtre glissante 5s
valeurs_en_temps_reel = [0.0 for _ in range(num_capteurs)] # Dernières valeurs
# Nouveau : pour gérer la fermeture propre des deux fenêtres
fenetres_ouvertes = 4 # nombre total de fenêtres à gérer

lock = threading.Lock()

# ----- THREAD DE LECTURE SERIE -----
def lire_serial():
    global acquisition_active
    while acquisition_active:
        try:
            line = ser.readline().decode(errors='ignore').strip()
            valeurs = list(map(float, line.split(",")))
            if len(valeurs) != num_capteurs:
```

```

        continue
    now = time.time()
    with lock:
        for i in range(num_capteurs):
            val = valeurs[i]
            valeurs_en_temps_reel[i] = val
            data_buffers[i].append(val)
            data_fenetre[i].append((now, val))
    except Exception:
        continue

thread_acq = threading.Thread(target=lire_serial, daemon=True)
thread_acq.start()

# Petit délai pour laisser l'acquisition série démarrer
# time.sleep(1)

# ----- FENETRE 1 : HISTOGRAMMES -----
fig_freq, axs_freq = plt.subplots(1, num_capteurs, figsize=(6 * num_capteurs, 4))
if num_capteurs == 1:
    axs_freq = [axs_freq]

def update_histogram(frame):
    with lock:
        now = time.time()
        snapshots = []
        for i in range(num_capteurs):
            if mode == "fenetre":
                while data_fenetre[i] and now - data_fenetre[i][0][0] > 5:
                    data_fenetre[i].popleft()
                valeurs = [v for (t, v) in data_fenetre[i]]
            else:
                valeurs = list(data_buffers[i])
            snapshots.append(valeurs)

    for i in range(num_capteurs):
        ax = axs_freq[i]
        ax.clear()
        ax.set_title(f'Capteur {i+1} - Mode: {mode}')
        ax.set_xlim(0, 100)
        ax.set_ylim(0, 50)
        ax.set_xlabel("Valeur")
        ax.set_ylabel("Fréquence")

        valeurs = snapshots[i]
        if valeurs:
            counts, bins = np.histogram(valeurs, bins=30, range=(0, 100))
            colors = plt.cm.jet(counts / counts.max()) if counts.max() > 0 else 'gray'
            ax.bar(bins[:-1], counts, width=(bins[1] - bins[0]), color=colors, align='edge')
        else:
            ax.text(0.5, 0.5, "Aucune donnée", ha='center', va='center', transform=ax.transAxes)

ani_freq = animation.FuncAnimation(fig_freq, update_histogram, interval=interval_update_sec * 1000)

# ----- FENETRE 2 : TEMPS REEL -----
fig_rt, ax_rt = plt.subplots()
bars = ax_rt.bar(labels, [0] * num_capteurs, color='skyblue')
ax_rt.set_ylim(0, 4096)

```

```
ax_rt.set_title("Valeurs en temps réel")
ax_rt.set_ylabel("Valeur")
text_labels = [ax_rt.text(i, 0, "", ha='center', va='bottom') for i in range(num_capteurs)]
```

```
def update_realtime(frame):
    with lock:
        snapshot = list(valeurs_en_temps_reel)
        for i, bar in enumerate(bars):
            val = snapshot[i]
            bar.set_height(val)
            text_labels[i].set_text(f'{val:.1f}')
            text_labels[i].set_y(val + 2)
```

```
ani_rt = animation.FuncAnimation(fig_rt, update_realtime, interval=100)
```

```
#-----Ferme le port série proprement
# Attache cette fonction aux deux fenêtres
fig_freq.canvas.mpl_connect("close_event", on_close)
fig_rt.canvas.mpl_connect("close_event", on_close)
```

```
# ----- FENETRE 3 : 3D MANCHE + CENTRE DES FORCES
fig_3d = plt.figure()
ax_3d = fig_3d.add_subplot(111, projection='3d')
fig_3d.canvas.mpl_connect("close_event", on_close)
```

```
# Coordonnées fixes des capteurs [x, y, z] en cm
capteurs_coords = np.array([
    [2, -1.5, 0.5], # Capteur 1
    [2, +1.5, 0.5], # Capteur 2
    [8, -1.5, 0.5], # Capteur 3
    [8, +1.5, 0.5], # Capteur 4
    [8, 0.0, 2.0], # Capteur 5
    [2, 0.0, 2.0], # Capteur 6
])
```

```
# Manche dimensions (pour affichage)
manche_length = 10
manche_width = 3
manche_height = 2
```

```
# Boîte du manche pour visuel
def draw_manche(ax):
    # Crée les coins d'une boîte
    from itertools import product, combinations
    r = [0, manche_length]
    w = [-manche_width/2, manche_width/2]
    h = [0, manche_height]
    points = list(product(r, w, h))
    for s, e in combinations(points, 2):
        if sum([s[i] != e[i] for i in range(3)]) == 1:
            ax.plot3D(*zip(s, e), color="gray", alpha=0.4)
```

```
def update_3d(frame):
    with lock:
        forces = np.array(valeurs_en_temps_reel)

    ax_3d.clear()
    draw_manche(ax_3d)
```

```

ax_3d.set_xlim(0, manche_length)
ax_3d.set_ylim(-manche_width/2 - 1, manche_width/2 + 1)
ax_3d.set_zlim(0, manche_height + 1)
ax_3d.set_title("Manche + Centre des forces")
ax_3d.set_xlabel("X (longueur)")
ax_3d.set_ylabel("Y (largeur)")
ax_3d.set_zlabel("Z (hauteur)")

# Affiche capteurs
for i, (x, y, z) in enumerate(capteurs_coords):
    ax_3d.scatter(x, y, z, color='blue')
    ax_3d.text(x, y, z + 0.2, f"{i+1}", color='blue')
# Calcul du centre de force
if np.sum(forces) > 0:
    center_force = np.average(capteurs_coords, axis=0, weights=forces)
    ax_3d.scatter(*center_force, color='red', s=100, label='Centre des forces')
    ax_3d.legend()

ani_3d = animation.FuncAnimation(fig_3d, update_3d, interval=200)

# ----- FENETRE 4 : AFFICHAGE DES 3 FLANCS -----
fig_color, ax_color = plt.subplots()
fig_color.canvas.mpl_connect("close_event", on_close)
fig_color.subplots_adjust(bottom=0.2)

# Rectangle + 2 zones par face (gauche, haut, droite)
rects = {}
positions = [0.2, 0.5, 0.8]
capteurs_zones = [(0, 2), (5, 4), (1, 3)] # Gauche, Haut, Droite

# Couleur personnalisée vert -> rouge
green_red = LinearSegmentedColormap.from_list("green_red", ["blue", "red"])

acquisition_en_cours = True
start_time = time.time()
donnees_10s = [[] for _ in range(num_capteurs)]

# Zones dessinées
for i, x in enumerate(positions):
    rects[f"{i}_top"] = ax_color.add_patch(Rectangle((x - 0.05, 0.6), 0.1, 0.3, color='gray'))
    rects[f"{i}_bot"] = ax_color.add_patch(Rectangle((x - 0.05, 0.2), 0.1, 0.3, color='gray'))

ax_color.set_xlim(0, 1)
ax_color.set_ylim(0, 1)
ax_color.axis('off')
ax_color.set_title("Zones de pression sur le manche (10s)")

# Bouton réinitialiser
ax_button = plt.axes([0.4, 0.05, 0.2, 0.075])
button = widgets.Button(ax_button, 'Réinitialiser')

def reset_acquisition(event):
    global acquisition_en_cours, start_time, donnees_10s
    acquisition_en_cours = True
    start_time = time.time()
    donnees_10s = [[] for _ in range(num_capteurs)]
    for r in rects.values():

```

```
        r.set_color("gray")

button.on_clicked(reset_acquisition)

def update_color(frame):
    global acquisition_en_cours
    if not acquisition_en_cours:
        return

    now = time.time()
    with lock:
        for i in range(num_capteurs):
            donnees_10s[i].append(valeurs_en_temps_reel[i])

    if now - start_time >= 10:
        acquisition_en_cours = False
        for idx, (top_id, bot_id) in enumerate(capteurs_zones):
            moyenne_top = np.mean(donnees_10s[top_id])
            moyenne_bot = np.mean(donnees_10s[bot_id])
            rects[f"{idx}_top"].set_color(green_red(moyenne_top / 100))
            rects[f"{idx}_bot"].set_color(green_red(moyenne_bot / 100))

ani_color = animation.FuncAnimation(fig_color, update_color, interval=200)

# ----- AFFICHAGE FINAL
plt.show()
```

Annexe C : Neural network code

```
import serial
import pandas as pd
import time
import os
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import joblib

PORT_SERIE = 'COM3'
BAUDRATE = 9600
N_CAPTEURS = 2
CSV_FILENAME = "donnees_manche_2_cpt.csv"
SCALER_FILENAME = "scaler_manche.pkl"
MODEL_FILENAME = "modele_manche.pkl"
TEMPS_ACQUISITION_PAR_POSITION = 60 # secondes

def initialiser_csv():
    if os.path.exists(CSV_FILENAME):
        choix = input("Souhaitez-vous réinitialiser le fichier CSV ? (o/n) : ").lower()
        if choix == 'o':
            os.remove(CSV_FILENAME)
            print("Fichier réinitialisé.")
        else:
            print("Les nouvelles données seront ajoutées au fichier existant.")
    else:
        print("Le fichier n'existe pas, il sera créé.")

def lire_donnees_serie(ser):
    try:
        ligne = ser.readline().decode('utf-8').strip()
        valeurs = ligne.split(',')
        if len(valeurs) == N_CAPTEURS:
            return [float(v) for v in valeurs]
    except Exception:
        pass
    return None

def acquisition_par_positions():
    try:
        ser = serial.Serial(PORT_SERIE, BAUDRATE, timeout=1)
        print("Connexion série ouverte.")
        time.sleep(2)

        position = 0
        data_total = []

        print("\nDébut de l'acquisition pour la position 0.")
        print("Appuyez sur Entrée pour passer à la position suivante.")

        while position <= 2:
            print(f"\n→ Acquisition pour la position {position}.")
```

```

input("→ Appuyez sur Entrée pour commencer l'acquisition pour cette position.")

try:
    duree = float(input(f" ⚠ Entrez la durée d'acquisition en secondes pour la position {position} : "))
except ValueError:
    print("Entrée invalide. Utilisation de 60 secondes par défaut.")
    duree = 60

start_time = time.time()
while True:
    if ser.in_waiting:
        donnees = lire_donnees_ser(ser)
        if donnees:
            donnees.append(position)
            data_total.append(donnees)
            print(f"Position {position} : {donnees[-1]}")

        if time.time() - start_time > duree:
            break

    position += 1

ser.close()

colonnes = [f"capteur_{i+1}" for i in range(N_CAPTEURS)] + ["classe"]
df = pd.DataFrame(data_total, columns=colonnes)

if os.path.exists(CSV_FILENAME):
    df.to_csv(CSV_FILENAME, mode='a', index=False, header=False)
else:
    df.to_csv(CSV_FILENAME, index=False)

print(f"\n 📁 Données enregistrées dans {CSV_FILENAME}")

except Exception as e:
    print(f"Erreur : {e}")

def entrainer_modele():
    if not os.path.exists(CSV_FILENAME):
        print("Aucune donnée d'entraînement trouvée.")
        return None, None

    df = pd.read_csv(CSV_FILENAME)
    X = df.iloc[:, :-1].values
    y = df.iloc[:, -1].values

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    joblib.dump(scaler, SCALER_FILENAME)

    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2)

    model = MLPClassifier(hidden_layer_sizes=(12, 12), max_iter=300)
    model.fit(X_train, y_train)
    print(f"Précision sur test : {model.score(X_test, y_test)*100:.2f}%")
    joblib.dump(model, MODEL_FILENAME)

```

```
print("🧠 Modèle entraîné et sauvegardé.")
return model, scaler
```

```
def prediction_temps_reel():
    if not os.path.exists(MODEL_FILENAME) or not os.path.exists(SCALER_FILENAME):
        model, scaler = entrainer_modele()
        if model is None:
            print("Impossible de lancer la prédiction sans modèle.")
            return
    else:
        model = joblib.load(MODEL_FILENAME)
        scaler = joblib.load(SCALER_FILENAME)
```

```
ser = serial.Serial(PORT_SERIE, BAUDRATE, timeout=1)
print("🖱️ Prédiction en temps réel. Appuyez sur Ctrl+C pour arrêter.\n")
```

```
try:
    while True:
        if ser.in_waiting:
            donnees = lire_donnees_serie(ser)
            if donnees:
                entree = scaler.transform([donnees])
                pred = model.predict(entree)[0]
                proba = np.max(model.predict_proba(entree)) * 100
                print(f"Prédiction : position {pred} (confiance : {proba:.1f}%)")
except KeyboardInterrupt:
    print("\n🛑 Arrêt des prédictions.")
    ser.close()
```

```
if __name__ == "__main__":
    mode = input("Choisissez le mode : (a)cquisition, (p)rédiction ou (e)ntraînement ? : ").lower()
    if mode == 'a':
        initialiser_csv()
        acquisition_par_positions()
    elif mode == 'p':
        prediction_temps_reel()
    elif mode == 'e':
        entrainer_modele()
    else:
        print("Mode inconnu.")
```

Annexe D : Flexible handle code

```
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import re

# Paramètres de connexion série
SERIAL_PORT = 'COM5' # À adapter
BAUDRATE = 115200

# Initialisation de la liaison série
ser = serial.Serial(SERIAL_PORT, BAUDRATE, timeout=1)

# Initialisation du graphique
fig, ax = plt.subplots()
bar_labels = [f'R{i+1}' for i in range(6)] # 6 capteurs
bar_values = [0] * 6
bars = ax.bar(bar_labels, bar_values, color='skyblue')
value_texts = [ax.text(i, 0, "", ha='center', va='bottom') for i in range(6)]

ax.set_ylim(0, 2000)
ax.set_title("Valeurs brutes des capteurs R1 à R6")
ax.set_ylabel("Valeur brute")

# Fonction pour parser une ligne de type : "R1:565 R2:343 ... R6:XXX BTN:0"
def parse_line(line):
    matches = re.findall(r'R(\d):(\d+)', line)
    values = [0] * 6
    for sensor, val in matches:
        index = int(sensor) - 1
        if 0 <= index < 6:
            values[index] = int(val)
    return values

# Mise à jour du graphique
def update(frame):
    try:
        line = ser.readline().decode('utf-8').strip()
        if line:
            values = parse_line(line)
            for i, val in enumerate(values):
                bars[i].set_height(val)
                value_texts[i].set_text(str(val))
                value_texts[i].set_y(val + 20)
    except Exception as e:
        print("Erreur :", e)

# Fonction appelée à la fermeture de la figure
def on_close(event):
    print("Fermeture de la fenêtre. Libération du port série...")
    if ser.is_open:
        ser.close()

# Connexion de l'événement de fermeture
fig.canvas.mpl_connect('close_event', on_close)

ani = animation.FuncAnimation(fig, update, interval=50)
plt.show()
```

Annexe E : Neural network program for flexible handle

```
import serial
import pandas as pd
import time
import os
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import joblib
import re

PORT_SERIE = 'COM4'
BAUDRATE = 115200
N_CAPTEURS = 6
CSV_FILENAME = "donnees_manche_souple.csv"
SCALER_FILENAME = "scaler_manche.pkl"
MODEL_FILENAME = "modele_manche.pkl"

def initialiser_csv():
    if os.path.exists(CSV_FILENAME):
        choix = input("Souhaitez-vous réinitialiser le fichier CSV ? (o/n) : ").lower()
        if choix == 'o':
            os.remove(CSV_FILENAME)
            print("Fichier réinitialisé.")
        else:
            print("Les nouvelles données seront ajoutées au fichier existant.")
    else:
        print("Le fichier n'existe pas, il sera créé.")

def lire_donnees_serie(ser):
    try:
        ligne = ser.readline().decode('utf-8').strip()
        matches = re.findall(r'R(\d+):(\d+)', ligne)
        values = [0] * N_CAPTEURS
        for sensor, val in matches:
            idx = int(sensor) - 1
            if 0 <= idx < N_CAPTEURS:
                values[idx] = int(val)
        return values
    except Exception:
        pass
    return None

def acquisition_par_positions():
    try:
        ser = serial.Serial(PORT_SERIE, BAUDRATE, timeout=1)
        print("Connexion série ouverte.")
        time.sleep(2)

        position = 0
        data_total = []

        print("\nDébut de l'acquisition pour la position 0.")
        print("Appuyez sur Entrée pour passer à la position suivante.")
```

```

while position <= 2:
    print(f"\n→ Acquisition pour la position {position}.")
    input("→ Appuyez sur Entrée pour commencer l'acquisition pour cette position.")

    try:
        duree = float(input(f" ⚠ Entrez la durée d'acquisition en secondes pour la position {position} : "))
    except ValueError:
        print("Entrée invalide. Utilisation de 60 secondes par défaut.")
        duree = 60

    start_time = time.time()
    while True:
        if ser.in_waiting:
            donnees = lire_donnees_ser(ser)
            if donnees:
                donnees.append(position)
                data_total.append(donnees)
                print(f"Position {position} : {donnees[-1]}")

            if time.time() - start_time > duree:
                break

        position += 1

ser.close()

colonnes = [f"capteur_{i+1}" for i in range(N_CAPTEURS)] + ["classe"]
df = pd.DataFrame(data_total, columns=colonnes)

if os.path.exists(CSV_FILENAME):
    df.to_csv(CSV_FILENAME, mode='a', index=False, header=False)
else:
    df.to_csv(CSV_FILENAME, index=False)

print(f"\n 📁 Données enregistrées dans {CSV_FILENAME}")

except Exception as e:
    print(f"Erreur : {e}")

def entrainer_modele():
    if not os.path.exists(CSV_FILENAME):
        print("Aucune donnée d'entraînement trouvée.")
        return None, None

    df = pd.read_csv(CSV_FILENAME)
    X = df.iloc[:, :-1].values
    y = df.iloc[:, -1].values

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    joblib.dump(scaler, SCALER_FILENAME)

    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2)

    model = MLPClassifier(hidden_layer_sizes=(12, 12), max_iter=300)

```

```

model.fit(X_train, y_train)
print(f"Précision sur test : {model.score(X_test, y_test)*100:.2f}%")
joblib.dump(model, MODEL_FILENAME)
print("🧠 Modèle entraîné et sauvegardé.")
return model, scaler

def prediction_temps_reel():
    if not os.path.exists(MODEL_FILENAME) or not os.path.exists(SCALER_FILENAME):
        model, scaler = entrainer_modele()
        if model is None:
            print("Impossible de lancer la prédiction sans modèle.")
            return
    else:
        model = joblib.load(MODEL_FILENAME)
        scaler = joblib.load(SCALER_FILENAME)

    ser = serial.Serial(PORT_SERIE, BAUDRATE, timeout=1)
    print("🖱️ Prédiction en temps réel. Appuyez sur Ctrl+C pour arrêter.\n")

    try:
        while True:
            if ser.in_waiting:
                donnees = lire_donnees_serie(ser)
                if donnees and len(donnees) == N_CAPTEURS:
                    entree = scaler.transform([donnees])
                    pred = model.predict(entree)[0]
                    proba = np.max(model.predict_proba(entree)) * 100
                    print(f"Prédiction : position {pred} (confiance : {proba:.1f}%)")
    except KeyboardInterrupt:
        print("\n🛑 Arrêt des prédictions.")
        ser.close()

if __name__ == "__main__":
    mode = input("Choisissez le mode : (a)cquisition, (p)rédiction ou (e)ntraînement ? : ").lower()
    if mode == 'a':
        initialiser_csv()
        acquisition_par_positions()
    elif mode == 'p':
        prediction_temps_reel()
    elif mode == 'e':
        entrainer_modele()
    else:
        print("Mode inconnu.")

```

Annexe F : Neural network for movements

```
import serial
import pandas as pd
import time
import os
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import joblib

# === Paramètres à personnaliser ===
DOSSIER_TRAVAIL = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA
SUDOE\tactile_sensor\Capteurs_tactiles\PPorPS"
PORT_SERIE = 'COM3'
BAUDRATE = 9600
N_CAPTEURS = 6
NB_VALEURS_GLISSANTES = 5 # Taille fenêtre glissante
HIDDEN_LAYER = (30, 30)
MAX_ITER = 2500
CLASSES = [
    "neutre",
    "rectiligne_std",
    "courbe_droite_std",
    "courbe_gauche_std",
    "rectiligne_poignard",
    "courbe_gauche_poignard",
    "courbe_droite_poignard"
]

CSV_FILENAME = os.path.join(DOSSIER_TRAVAIL, "donnees_mouvements_UR5_statique.csv")
SCALER_FILENAME = os.path.join(DOSSIER_TRAVAIL, "scaler.pkl")
MODEL_FILENAME = os.path.join(DOSSIER_TRAVAIL, "modele.pkl")

# === Fonctions ===
def lire_donnees_serie(ser, num_capteurs):
    """Lit une ligne du port série et extrait les valeurs des capteurs."""
    try:
        line = ser.readline().decode(errors='ignore').strip()
        valeurs = list(map(float, line.split(",")))
        if len(valeurs) != num_capteurs:
            return None
        return valeurs
    except Exception:
        return None

def initialiser_csv():
    if os.path.exists(CSV_FILENAME):
        choix = input("Réinitialiser le fichier CSV ? (o/n) : ").lower()
        if choix == 'o':
            os.remove(CSV_FILENAME)
            print("Fichier CSV réinitialisé.")
        else:
            print("Les nouvelles données seront ajoutées.")
    else:
        print("Le fichier sera créé.")

def acquisition_par_classe():
```

```

ser = serial.Serial(PORT_SERIE, BAUDRATE, timeout=1)
print("Connexion série établie.")
time.sleep(2)

data_total = []

for classe in CLASSES:
    input(f"\nPréparez la classe '{classe}'. Appuyez sur Entrée pour démarrer l'acquisition.")
    try:
        duree = float(input("Durée d'acquisition (sec) : "))
    except ValueError:
        duree = 60
    print("Durée par défaut : 60s.")

    buffer = []
    start = time.time()
    while time.time() - start < duree:
        if ser.in_waiting:
            donnees = lire_donnees_serie(ser, N_CAPTEURS)
            if donnees:
                buffer.append(donnees)
                if len(buffer) >= NB_VALEURS_GLISSANTES:
                    fenetre = buffer[-NB_VALEURS_GLISSANTES:]
                    ligne = sum(fenetre, [])
                    ligne.append(classe)
                    data_total.append(ligne)
                    print(f"{classe} : {ligne[:-1]}")

    print(f"Acquisition terminée pour {classe}.")

ser.close()
colonnes = [f"capteur_{i+1}_t{j+1}" for j in range(NB_VALEURS_GLISSANTES) for i in range(N_CAPTEURS)] +
["classe"]
df = pd.DataFrame(data_total, columns=colonnes)

if os.path.exists(CSV_FILENAME):
    df.to_csv(CSV_FILENAME, mode='a', index=False, header=False)
else:
    df.to_csv(CSV_FILENAME, index=False)

print(f"✅ Données enregistrées dans {CSV_FILENAME}")

def entrainer_modele(HIDDEN_LAYER, MAX_ITER):
    if not os.path.exists(CSV_FILENAME):
        print("Fichier CSV non trouvé.")
        return

    df = pd.read_csv(CSV_FILENAME)
    X = df.iloc[:, :-1].values
    y = df.iloc[:, -1].values

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    joblib.dump(scaler, SCALER_FILENAME)

    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2)

```

```

        model = MLPClassifier(hidden_layer_sizes=HIDDEN_LAYER, max_iter=MAX_ITER, solver='adam',
activation='logistic', verbose=True)
        model.fit(X_train, y_train)

    print(f"🎯 Précision : {model.score(X_test, y_test)*100:.2f}%")
    joblib.dump(model, MODEL_FILENAME)
    print("🧠 Modèle sauvegardé.")

def prediction_temps_reel():
    if not os.path.exists(MODEL_FILENAME) or not os.path.exists(SCALER_FILENAME):
        print("Modèle non trouvé.")
        return

    model = joblib.load(MODEL_FILENAME)
    scaler = joblib.load(SCALER_FILENAME)
    ser = serial.Serial(PORT_SERIE, BAUDRATE, timeout=1)
    print("🖱️ Prédiction en cours. Ctrl+C pour arrêter.")

    buffer = []

    try:
        while True:
            if ser.in_waiting:
                donnees = lire_donnees_serie(ser, N_CAPTEURS)
                if donnees:
                    buffer.append(donnees)
                    if len(buffer) >= NB_VALEURS_GLISSANTES:
                        fenetre = buffer[-NB_VALEURS_GLISSANTES:]
                        entree = sum(fenetre, [])
                        X_input = scaler.transform([entree])
                        pred = model.predict(X_input)[0]
                        proba = np.max(model.predict_proba(X_input)) * 100
                        print(f"Prédiction : {pred} ( {proba:.1f}% )")
    except KeyboardInterrupt:
        print("🛑 Arrêt.")
        ser.close()

# === Menu principal ===
if __name__ == "__main__":
    mode = input("Mode : (a)cquisition, (e)ntrainement, (p)rédiction ? ").lower()
    if mode == 'a':
        initialiser_csv()
        acquisition_par_classe()
    elif mode == 'e':
        entrainer_modele(HIDDEN_LAYER, MAX_ITER)
    elif mode == 'p':
        prediction_temps_reel()
    else:
        print("Mode inconnu.")

```

Annexe G : 3D simulation of the handle in real time

```
import serial
import time
import numpy as np
import matplotlib.pyplot as plt
import pyvista as pv
from pyvistaqt import BackgroundPlotter
from stl import mesh
import threading
from queue import Queue

# --- Paramètres de configuration ---
SERIAL_PORT = 'COM3'
BAUD_RATE = 9600
STL_FILE_PATH = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA SUDOE\ROBOTA SUDOE 2025 BRIANCON-PROVERBIO\CAD\STL\Handle\Manche_couteau.STL"

rect_params = [
    {'x': 20, 'y': 20, 'z': 31, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 20, 'y': 20, 'z': 7, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 85, 'y': 20, 'z': 31, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 85, 'y': 20, 'z': 7, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 85, 'y': 45, 'z': 8.8, 'width': 60, 'height': 1, 'depth': 20},
    {'x': 20, 'y': 45, 'z': 8.8, 'width': 60, 'height': 1, 'depth': 20}
]

# File d'attente pour la communication entre les threads
data_queue = Queue()
stop_thread = threading.Event()

# --- Fonction de lecture du port série dans un thread séparé ---
def read_serial_data():
    """Lit les données du port série et met à jour la file d'attente."""
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.1)
        time.sleep(2)
        ser.reset_input_buffer()
        print(f"Thread de lecture série démarré sur {SERIAL_PORT}")

        while not stop_thread.is_set():
            # Vidage du buffer si des données s'y accumulent
            if ser.in_waiting > 0:
                ser.reset_input_buffer()

            line = ser.readline().decode('utf-8').strip()
            if line:
                try:
                    new_values = [float(val) for val in line.split(',') if val]
                    if len(new_values) == 6:
                        data_queue.put(new_values)
                except (ValueError, IndexError) as e:
                    print(f"Erreur de format de données: {line} - {e}")
                time.sleep(0.01)

    except serial.SerialException as e:
        print(f"Erreur fatale dans le thread série: {e}")
    finally:
        if 'ser' in locals() and ser.is_open:
```

```

        ser.close()

# --- Fonction principale ---
def main():
    serial_thread = threading.Thread(target=read_serial_data, daemon=True)
    serial_thread.start()

    plotter = BackgroundPlotter(title="Modélisation 3D du manche de couteau")

    try:
        pyvista_mesh = pv.read(STL_FILE_PATH)
        plotter.add_mesh(pyvista_mesh, color='white', smooth_shading=True)
    except FileNotFoundError:
        print(f"Erreur: Le fichier STL '{STL_FILE_PATH}' est introuvable.")
        stop_thread.set()
        serial_thread.join()
        return

    sensors_actors = []
    for params in rect_params:
        box = pv.Box(bounds=(params['x'], params['x'] + params['width'],
                             params['y'], params['y'] + params['height'],
                             params['z'], params['z'] + params['depth']))
        actor = plotter.add_mesh(box, color='blue', opacity=0.5)
        sensors_actors.append(actor)

    color_map = plt.get_cmap("bwr")
    norm = plt.Normalize(vmin=0, vmax=4096)

    def update_colors():
        last_values = None
        while not data_queue.empty():
            last_values = data_queue.get_nowait()

        if last_values:
            print(f"Données mises à jour: {last_values}")
            try:
                for i, value in enumerate(last_values):
                    color = color_map(norm(value))
                    sensors_actors[i].prop.color = color
            except Exception as e:
                print(f"Erreur lors de la mise à jour des couleurs: {e}")

    plotter.add_callback(update_colors, interval=50)

    try:
        plotter.app.exec()
    except KeyboardInterrupt:
        print("Fermeture de l'application.")
    finally:
        stop_thread.set()
        serial_thread.join()
        if 'ser' in locals() and ser.is_open:
            ser.close()
        plotter.close()

if __name__ == '__main__':
    main()

```

Annexe H : 3D simulation of the handle with csv files

```
import pandas as pd
import numpy as np
import pyvista as pv
from pyvistaqt import BackgroundPlotter
import matplotlib.pyplot as plt
import os
import time

# --- Paramètres de configuration (à ajuster) ---
CSV_FILE_PATH = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA SUDOE\Tactile_sensor\ROBOTA_SUDOE\Plot_pos\wrench_camera_data\wrench_camera_data\linear_y_hp1\wrench_data_2025-08-04_17-43-46.csv"
STL_FILE_PATH = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA SUDOE\ROBOTA SUDOE 2025 BRIANCON-PROVERBIO\CAD\STL\Handle\Manche_couteau.STL"

# Correspondance entre les capteurs et les colonnes du fichier CSV
sensor_mapping = [21, 22, 23, 24, 25, 26]

rect_params = [
    {'x': 20, 'y': 20, 'z': 31, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 20, 'y': 20, 'z': 7, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 85, 'y': 20, 'z': 31, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 85, 'y': 20, 'z': 7, 'width': 60, 'height': 20, 'depth': 1},
    {'x': 85, 'y': 45, 'z': 8.8, 'width': 60, 'height': 1, 'depth': 20},
    {'x': 20, 'y': 45, 'z': 8.8, 'width': 60, 'height': 1, 'depth': 20}
]

# --- Paramètres de l'animation (à ajuster) ---
ANIMATION_DURATION_SECONDS = 30 # Durée totale de l'animation en secondes

# --- Variables de contrôle de l'animation ---
animation_running = False
current_frame_index = 0
data_to_plot = None
sensors_actors = []
color_map = None
norm = None

# --- Fonction de rappel pour l'animation ---
def update_animation():
    global current_frame_index, animation_running

    if not animation_running or data_to_plot is None:
        return

    if current_frame_index < len(data_to_plot):
        try:
            current_values = data_to_plot.iloc[current_frame_index, :].fillna(0).tolist()

            # Mise à jour des couleurs des capteurs
            for i, value in enumerate(current_values):
                color = color_map(norm(value))
                sensors_actors[i].prop.color = color

            print(f"Frame {current_frame_index + 1}/{len(data_to_plot)} - Valeurs: {current_values}")

            current_frame_index += 1
```

```

except Exception as e:
    print(f"Erreur lors de la mise à jour de l'animation: {e}")
    animation_running = False

else:
    print("Animation terminée. Appuyez sur Espace pour recommencer.")
    current_frame_index = 0
    animation_running = False

# --- Fonction pour démarrer/arrêter l'animation ---
def toggle_animation():
    global animation_running
    animation_running = not animation_running
    if animation_running:
        print("Animation démarrée. Appuyez sur Espace pour mettre en pause.")
    else:
        print("Animation en pause. Appuyez sur Espace pour reprendre.")

# --- Fonction principale ---
def main():
    global data_to_plot, sensors_actors, color_map, norm

    try:
        # 1. Lecture des données à partir du fichier CSV
        df = pd.read_csv(CSV_FILE_PATH, header=None).apply(pd.to_numeric, errors='coerce')
        data_to_plot = df.iloc[:, sensor_mapping]

        num_frames = len(data_to_plot)
        interval_ms = int(np.ceil((ANIMATION_DURATION_SECONDS * 1000) / num_frames))

        print(f"Total de données chargées : {num_frames}.")
        print(f"Intervalle d'affichage calculé : {interval_ms} ms par frame.")
        print(f"Durée totale de l'animation : {ANIMATION_DURATION_SECONDS} secondes.")
        print("Appuyez sur la touche ESPACE pour démarrer l'animation.")

    except FileNotFoundError:
        print(f"Erreur: Le fichier CSV '{CSV_FILE_PATH}' est introuvable.")
        return

    except Exception as e:
        print(f"Erreur lors de la lecture du fichier CSV : {e}")
        return

    # 2. Initialisation du plotter PyVista
    plotter = BackgroundPlotter(title="Modélisation 3D du manche de couteau")

    try:
        pyvista_mesh = pv.read(STL_FILE_PATH)
        plotter.add_mesh(pyvista_mesh, color='white', smooth_shading=True)
    except FileNotFoundError:
        print(f"Erreur: Le fichier STL '{STL_FILE_PATH}' est introuvable.")
        plotter.close()
        return

    for params in rect_params:
        box = pv.Box(bounds=(params['x'], params['x'] + params['width'],
                             params['y'], params['y'] + params['height'],
                             params['z'], params['z'] + params['depth']))

```

```
    actor = plotter.add_mesh(box, color='blue', opacity=0.5)
    sensors_actors.append(actor)

color_map = plt.get_cmap("bwr")
norm = plt.Normalize(vmin=0, vmax=4096)

# 3. Ajout du contrôle de l'animation
plotter.add_key_event('space', toggle_animation)

# Exécuter la fonction de mise à jour à l'intervalle calculé
plotter.add_callback(update_animation, interval=interval_ms)

try:
    plotter.app.exec()
except KeyboardInterrupt:
    print("Fermeture de l'application.")
finally:
    plotter.close()

if __name__ == '__main__':
    main()
```

Annexe I : Calculation of the Jacobian

import numpy as np

Paramètres de Denavit-Hartenberg (DH)

Les longueurs et offsets sont en mm, les angles en degrés.

a = np.array([0, 638, 502, 13, 0])

d = np.array([237.0, 0, 0, 0, -201.0])

alpha = np.array([90, 0, 0, 90, -90])

def calculate_transform_matrix(a_i, d_i, alpha_i, theta_i):

"""

Calculates the transformation matrix for a single joint using DH parameters.

"""

theta_rad = np.deg2rad(theta_i)

alpha_rad = np.deg2rad(alpha_i)

```
T = np.array([
    [np.cos(theta_rad), -np.sin(theta_rad) * np.cos(alpha_rad), np.sin(theta_rad) * np.sin(alpha_rad), a_i *
np.cos(theta_rad)],
    [np.sin(theta_rad), np.cos(theta_rad) * np.cos(alpha_rad), -np.cos(theta_rad) * np.sin(alpha_rad), a_i *
np.sin(theta_rad)],
    [0, np.sin(alpha_rad), np.cos(alpha_rad), d_i],
    [0, 0, 0, 1]
])
return T
```

def calculate_forward_kinematics_matrix(q, a, d, alpha):

"""

Calculates the final transformation matrix for the end-effector.

"""

T_final = np.identity(4)

for i in range(len(q)):

 T_link = calculate_transform_matrix(a[i], d[i], alpha[i], q[i])

 T_final = np.dot(T_final, T_link)

return T_final

def calculate_jacobian(q, a, d, alpha):

"""

Calculates the geometric Jacobian matrix.

"""

n_dof = len(q)

J = np.zeros((6, n_dof))

T_list = [np.identity(4)]

T_current = np.identity(4)

for i in range(n_dof):

 T_current = np.dot(T_current, calculate_transform_matrix(a[i], d[i], alpha[i], q[i]))

 T_list.append(T_current)

p_end_effector = T_list[-1][:3, 3]

for i in range(n_dof):

 T_i_minus_1 = T_list[i]

 z_i_minus_1 = T_i_minus_1[:3, 2]

 p_i_minus_1 = T_i_minus_1[:3, 3]

 J[3:, i] = np.cross(z_i_minus_1, p_end_effector - p_i_minus_1)

 J[3:, i] = z_i_minus_1

return J

Annexe J : Manipulability ellipse code

```
import pyvista as pv
from pyvistaqt import QtInteractor
from PyQt5 import QtWidgets, QtCore
import sys
import os
import numpy as np

# Ajoute le répertoire où se trouve robot_kinematics.py
chemin_du_repertoire = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA
SUDOE\Tactile_sensor\ROBOTA_SUDOE\Plot_pos\Ellipse"
sys.path.append(os.path.abspath(chemin_du_repertoire))

from robot_kinematics import calculate_jacobian, a, d, alpha

# Chemins de vos fichiers STL
base_path = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA
SUDOE\Tactile_sensor\ROBOTA_SUDOE\Plot_pos\Ellipse\704-244-01_filled_A\Base.stl"
shoulder_path = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA
SUDOE\Tactile_sensor\ROBOTA_SUDOE\Plot_pos\Ellipse\704-244-01_filled_A\Shoulder.stl"
elbow_path = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA
SUDOE\Tactile_sensor\ROBOTA_SUDOE\Plot_pos\Ellipse\704-244-01_filled_A\Elbow.stl"
wrist1_path = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA
SUDOE\Tactile_sensor\ROBOTA_SUDOE\Plot_pos\Ellipse\704-244-01_filled_A\Wrist1.stl"
wrist2_path = r"C:\Users\matte\OneDrive\Documents\Scolaire\Sigma\2A\Stage\ROBOTA
SUDOE\Tactile_sensor\ROBOTA_SUDOE\Plot_pos\Ellipse\704-244-01_filled_A\Wrist2.stl"

# Définition des points de pivot (inchangé)
link_pivots = {
    'Shoulder': np.array([0.0, 237.0, -135.0]),
    'Elbow': np.array([0.0, 875.0, 0.0]),
    'Wrist 1': np.array([0.0, 1377.0, 0.0]),
    'Wrist 2': np.array([0.0, 1390.0, -201.0]),
}

# Charger et décaler les maillages
original_meshes = {
    'Base': pv.read(base_path),
    'Shoulder': pv.read(shoulder_path),
    'Elbow': pv.read(elbow_path),
    'Wrist 1': pv.read(wrist1_path),
    'Wrist 2': pv.read(wrist2_path)
}

shoulder_offset = -link_pivots['Shoulder']
elbow_offset = -link_pivots['Elbow']
wrist1_offset = -link_pivots['Wrist 1']
wrist2_offset = -link_pivots['Wrist 2']

initial_meshes = {
    'Base': original_meshes['Base'],
    'Shoulder': original_meshes['Shoulder'].translate(shoulder_offset),
    'Elbow': original_meshes['Elbow'].translate(elbow_offset),
    'Wrist 1': original_meshes['Wrist 1'].translate(wrist1_offset),
    'Wrist 2': original_meshes['Wrist 2'].translate(wrist2_offset)
}

class RobotApp(QtWidgets.QMainWindow):
```

```

def __init__(self, parent=None):
    super(RobotApp, self).__init__(parent)
    self.setWindowTitle("Contrôle du robot avec PyVista et PyQt")

    self.joint_angles = {
        'Base': 0.0, 'Shoulder': 45.0, 'Elbow': 45.0, 'Wrist 1': 0.0, 'Wrist 2': 0.0
    }

    main_widget = QtWidgets.QWidget()
    self.setCentralWidget(main_widget)
    main_layout = QtWidgets.QHBoxLayout(main_widget)

    self.plotter = QtInteractor(self)

    control_panel = QtWidgets.QWidget()
    control_layout = QtWidgets.QVBoxLayout(control_panel)

    splitter = QtWidgets.QSplitter(QtCore.Qt.Horizontal)
    splitter.addWidget(self.plotter)
    splitter.addWidget(control_panel)

    splitter.setStretchFactor(0, 3)
    splitter.setStretchFactor(1, 1)

    main_layout.addWidget(splitter)

    controllable_joints = ['Base', 'Shoulder', 'Elbow', 'Wrist 1', 'Wrist 2']

    self.entries = {}
    for name in controllable_joints:
        entry_layout = QtWidgets.QHBoxLayout()
        label = QtWidgets.QLabel(f'Angle {name}:')
        entry = QtWidgets.QLineEdit(str(self.joint_angles[name]))
        entry.returnPressed.connect(lambda n=name, e=entry: self.on_text_entry(n, e))
        self.entries[name] = entry
        entry_layout.addWidget(label)
        entry_layout.addWidget(entry)
        control_layout.addLayout(entry_layout)

    self.manip_ellipsoid_actor = None
    self.effector_marker = None
    self.setup_robot_scene()

def setup_robot_scene(self):
    self.plotter.background_color = 'white'
    self.robot_actors = {
        'Base': self.plotter.add_mesh(initial_meshes['Base'], color='lightblue', label='Base'),
        'Shoulder': self.plotter.add_mesh(initial_meshes['Shoulder'], color='lightgreen', label='Shoulder'),
        'Elbow': self.plotter.add_mesh(initial_meshes['Elbow'], color='salmon', label='Elbow'),
        'Wrist 1': self.plotter.add_mesh(initial_meshes['Wrist 1'], color='gold', label='Wrist 1'),
        'Wrist 2': self.plotter.add_mesh(initial_meshes['Wrist 2'], color='lightcoral', label='Wrist 2')
    }

    self.pivot_meshes = {}
    for name, pos in link_pivots.items():
        self.pivot_meshes[name] = pv.Sphere(radius=5).translate(pos)
        self.plotter.add_mesh(self.pivot_meshes[name], color='red', render_points_as_spheres=True, point_size=10.0)

```

```

slider_params = [
    ('Base', [-180, 180], self.joint_angles['Base']),
    ('Shoulder', [-180, 180], self.joint_angles['Shoulder']),
    ('Elbow', [-180, 180], self.joint_angles['Elbow']),
    ('Wrist 1', [-180, 180], self.joint_angles['Wrist 1']),
    ('Wrist 2', [-180, 180], self.joint_angles['Wrist 2'])
]
spacing = 0.15
start_y = 0.1
for i, (name, rng, value) in enumerate(slider_params):
    self.plotter.add_slider_widget(
        callback=lambda val, n=name: self.on_slider_change(val, n),
        rng=rng, title=name, pointa=(0.02, start_y + i * spacing), pointb=(0.32, start_y + i * spacing),
        value=value, style='modern'
    )

self.update_robot()

def update_robot(self):
    t_base = pv.Transform().rotate_y(self.joint_angles['Base'])
    t_shoulder = pv.Transform().rotate_z(self.joint_angles['Shoulder'])
    t_elbow = pv.Transform().rotate_z(self.joint_angles['Elbow'])
    t_wrist1 = pv.Transform().rotate_z(self.joint_angles['Wrist 1'])
    t_wrist2 = pv.Transform().rotate_y(self.joint_angles['Wrist 2'])

    final_transform_base = t_base.matrix
    final_transform_shoulder = final_transform_base @ pv.Transform().translate(link_pivots['Shoulder']).matrix @
t_shoulder.matrix
    final_transform_elbow = final_transform_shoulder @ pv.Transform().translate(link_pivots['Elbow'] -
link_pivots['Shoulder']).matrix @ t_elbow.matrix
    final_transform_wrist1 = final_transform_elbow @ pv.Transform().translate(link_pivots['Wrist 1'] -
link_pivots['Elbow']).matrix @ t_wrist1.matrix
    final_transform_wrist2 = final_transform_wrist1 @ pv.Transform().translate(link_pivots['Wrist 2'] - link_pivots['Wrist
1']).matrix @ t_wrist2.matrix

    self.robot_actors['Base'].user_matrix = final_transform_base
    self.robot_actors['Shoulder'].user_matrix = final_transform_shoulder
    self.robot_actors['Elbow'].user_matrix = final_transform_elbow
    self.robot_actors['Wrist 1'].user_matrix = final_transform_wrist1
    self.robot_actors['Wrist 2'].user_matrix = final_transform_wrist2

    q_angles_deg = np.array(list(self.joint_angles.values()))

    try:
        jacobien = calculate_jacobian(q_angles_deg, a, d, alpha)

        wrist2_transform_matrix = self.robot_actors['Wrist 2'].user_matrix
        self.calculate_and_plot_ellipsoid(jacobien, wrist2_transform_matrix, scale_factor=250)
    except Exception as e:
        print(f"Erreur lors du calcul de la cinématique ou de l'affichage de l'ellipsoïde : {e}")

    self.plotter.render()

def on_slider_change(self, value, name):
    self.joint_angles[name] = value
    self.entries[name].setText(f" {value:.2f} ")
    self.update_robot()

```

```

def on_text_entry(self, name, entry):
    try:
        value = float(entry.text())
        self.joint_angles[name] = value
        self.update_robot()
    except ValueError:
        print(f"Valeur invalide pour {name}. Veuillez entrer un nombre.")

def calculate_and_plot_ellipsoid(self, jacobien, wrist2_transform_matrix, scale_factor=250):
    Jv = jacobien[:3, :]

    try:
        U, s, Vt = np.linalg.svd(Jv)

        if np.isclose(s[0], 0, atol=1e-6):
            if self.manip_ellipsoid_actor:
                self.plotter.remove_actor(self.manip_ellipsoid_actor)
                print("Le robot est dans une configuration singulière. L'ellipsoïde de manipulabilité ne peut pas être affiché.")
                return

            s_scaled = s / s[0] * scale_factor

            if self.manip_ellipsoid_actor:
                self.plotter.remove_actor(self.manip_ellipsoid_actor)

            ellipsoid_mesh = pv.Sphere(radius=1.0, phi_resolution=30, theta_resolution=30)

            # Paramètre pour le positionnement manuel de l'ellipse
            # decalage_local_ellipse = np.array([0.0, 1535.0, -300.0])
            decalage_local_ellipse = np.array([0.0, 150.0, -100.0])

            ellipsoid_mesh.scale(s_scaled, inplace=True)
            ellipsoid_mesh.translate(decalage_local_ellipse, inplace=True)
            ellipsoid_mesh.transform(wrist2_transform_matrix, inplace=True)

            self.manip_ellipsoid_actor = self.plotter.add_mesh(ellipsoid_mesh, color='red', opacity=0.5, style='wireframe')

        except np.linalg.LinAlgError:
            if self.manip_ellipsoid_actor:
                self.plotter.remove_actor(self.manip_ellipsoid_actor)
                print("Erreur de calcul SVD : Le robot est dans une configuration singulière.")
        except Exception as e:
            print(f"Erreur lors de l'affichage de l'ellipsoïde : {e}")

if __name__ == '__main__':
    app = QtWidgets.QApplication(sys.argv)
    window = RobotApp()
    window.show()
    sys.exit(app.exec_())

```

Annexe K : Code to move the ur5e in gazebo with direct kinematic

```
#!/usr/bin/env python3
import rospy
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
import time

def move_ur30_base_trajectory():
    """
    Ce script envoie une trajectoire en deux étapes pour faire pivoter le bras UR30.
    1. Retour à la position de départ (home).
    2. Rotation de la base de 90 degrés.
    """

    # Initialise le noeud ROS
    rospy.init_node('ur30_base_trajectory_controller', anonymous=True)

    # Crée un publisher pour envoyer les commandes de trajectoire
    pub = rospy.Publisher('/eff_joint_traj_controller/command', JointTrajectory, queue_size=10)

    # Attend que le publisher soit connecté au contrôleur
    rospy.sleep(1)

    # Crée le message de trajectoire
    traj_msg = JointTrajectory()

    # Définit le nom des joints
    traj_msg.joint_names = ['shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint',
                           'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint']

    # --- Point 1 : Retour à la position de départ (home) ---
    point1 = JointTrajectoryPoint()
    point1.positions = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    point1.time_from_start = rospy.Duration(3.0) # Temps pour atteindre la position home (3 secondes)

    # --- Point 2 : Rotation de la base de 90 degrés ---
    point2 = JointTrajectoryPoint()
    point2.positions = [0.0, -0.78, 1.57, -2.25, -1.57, 0.0]
    point2.time_from_start = rospy.Duration(6.0) # Temps total pour le mouvement (3s + 3s = 6s)

    # Ajoute les points à la trajectoire
    traj_msg.points.append(point1)
    traj_msg.points.append(point2)

    rospy.loginfo("Début de la séquence : retour à home puis rotation de 90 degrés.")

    # Publie le message de trajectoire
    pub.publish(traj_msg)

    # Attend la fin du mouvement
    rospy.sleep(6.5)

    rospy.loginfo("Séquence terminée.")

if __name__ == '__main__':
    try:
        move_ur30_base_trajectory()
    except rospy.ROSInterruptException:
        pass
```

Annexe L : Code to move the ur5e in gazebo with inverse kinematic

```
#!/usr/bin/env python3
import rospy
import sys
from geometry_msgs.msg import PoseStamped
from moveit_commander import RobotCommander, MoveGroupCommander, PlanningSceneInterface
from moveit_commander import roscpp_initialize, roscpp_shutdown
from moveit_msgs.msg import DisplayTrajectory
from tf.transformations import quaternion_from_euler

def move_to_pose(x, y, z, roll, pitch, yaw):
    """
    Planifie et exécute un mouvement pour que l'effecteur atteigne une pose donnée.
    """
    try:
        # Initialisation de moveit_commander et du noeud ROS
        roscpp_initialize(sys.argv)
        rospy.init_node('IK_position_moveit', anonymous=True)

        # Création des objets MoveIt
        robot = RobotCommander()
        scene = PlanningSceneInterface()
        group_name = "manipulator"
        move_group = MoveGroupCommander(group_name)

        # Définir le nom de la trame de référence pour la pose
        move_group.set_pose_reference_frame("base_link")

        # Créer l'objet PoseStamped pour la cible
        pose_goal = PoseStamped()
        pose_goal.header.frame_id = "base_link"
        pose_goal.header.stamp = rospy.Time.now()

        # Remplir les coordonnées de position
        pose_goal.pose.position.x = x
        pose_goal.pose.position.y = y
        pose_goal.pose.position.z = z

        # Convertir les angles d'Euler en Quaternions pour l'orientation
        q = quaternion_from_euler(roll, pitch, yaw)
        pose_goal.pose.orientation.x = q[0]
        pose_goal.pose.orientation.y = q[1]
        pose_goal.pose.orientation.z = q[2]
        pose_goal.pose.orientation.w = q[3]

        # Définir la cible de pose
        move_group.set_pose_target(pose_goal)

        # Planifier et exécuter le mouvement avec la fonction go()
        rospy.loginfo("Planification et exécution du mouvement...")
        success = move_group.go(wait=True)

        if success:
            rospy.loginfo("Mouvement réussi.")
        else:
            rospy.logerr("Le plan de mouvement a échoué. Assurez-vous que les coordonnées de la cible sont atteignables.")

        # Arrêter le mouvement et nettoyer la cible
```

```
    move_group.stop()
    move_group.clear_pose_targets()

except rospy.ROSInterruptException:
    pass
finally:
    # Arrêter le gestionnaire C++ de ROS
    roscpp_shutdown()

if __name__ == '__main__':
    # Définir les coordonnées de la cible
    # Attention: ces valeurs doivent être physiquement atteignables par l'UR5e
    x_target = 0.5
    y_target = 0.5
    z_target = 0.6
    roll_target = 0.78    # Orientation de l'effecteur en radians
    pitch_target = 0.0    # 90 degrés, pour que l'effecteur pointe vers le bas
    yaw_target = 0.0

    try:
        move_to_pose(x_target, y_target, z_target, roll_target, pitch_target, yaw_target)
    except Exception as e:
        rospy.logerr("Une erreur s'est produite : %s" % e)
```

Annexe M : Joystick publisher for direct kinematic

```
#!/usr/bin/env python
```

```
import rospy
from sensor_msgs.msg import Joy
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
from sensor_msgs.msg import JointState

class JoystickToMoveItPublisher:
    def __init__(self):
        rospy.init_node('joystick_publisher', anonymous=True)

        # Publisher for the MoveIt! joint position command topic
        # self.traj_pub = rospy.Publisher('/scaled_pos_joint_traj_controller/command', JointTrajectory, queue_size=1)
        self.traj_pub = rospy.Publisher('/eff_joint_traj_controller/command', JointTrajectory, queue_size=1)

        # Subscriber for raw joystick data
        rospy.Subscriber('/joy1', Joy, self.joy_callback)

        # Subscriber for the robot's current joint state
        rospy.Subscriber('/joint_states', JointState, self.joint_state_callback)

        # Joystick data
        self.joy_axes = [0.0] * 8
        self.joy_buttons = [0] * 11
        self.current_joint_positions = [0.0] * 6

        # Set the publishing rate
        self.rate = rospy.Rate(250) # 250 Hz for smooth motion

    def joint_state_callback(self, data):
        self.current_joint_positions = data.position

    def joy_callback(self, data):
        # Callback function simply updates the class variables
        self.joy_axes = data.axes
        self.joy_buttons = data.buttons

    def run(self):
        rospy.loginfo("Contrôleur démarré. Attente des données du joystick...")
        # Define delta_t once, based on the fixed rate
        delta_t = 2

        while not rospy.is_shutdown():
            if len(self.joy_axes) < 6 or len(self.joy_buttons) < 4:
                self.rate.sleep()
                continue

            # Joint speed factor
            joint_speed_factor = 0.5

            # Map joystick axes to joint velocities
            joint_velocities = [
                0,
                self.joy_axes[0] * joint_speed_factor,
                (self.joy_axes[1]-0.56) * joint_speed_factor,
                self.joy_axes[4] * joint_speed_factor,
```

```

        self.joy_axes[5] * joint_speed_factor,
        0,
    ]

    # Button logic
    if self.joy_buttons[0] == 1:
        joint_velocities[0] = joint_speed_factor
    if self.joy_buttons[1] == 1:
        joint_velocities[0] = -joint_speed_factor
    if self.joy_buttons[2] == 1:
        joint_velocities[5] = joint_speed_factor
    if self.joy_buttons[3] == 1:
        joint_velocities[5] = -joint_speed_factor

    if abs(joint_velocities[2]) < 0.2:
        joint_velocities[2] = 0

    # Calculate new target positions
    target_positions = [
        self.current_joint_positions[i] + joint_velocities[i] * delta_t for i in range(6)
    ]

    # Create and publish trajectory message
    traj_msg = JointTrajectory()
    traj_msg.joint_names = ['shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint',
                           'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint']

    point = JointTrajectoryPoint()
    point.positions = target_positions
    point.velocities = joint_velocities
    point.time_from_start = rospy.Duration(delta_t)

    traj_msg.points.append(point)
    self.traj_pub.publish(traj_msg)

    # Sleep to maintain the fixed rate
    self.rate.sleep()

if __name__ == '__main__':
    try:
        publisher = JoystickToMoveItPublisher()
        publisher.run()
    except rospy.ROSInterruptException:
        pass

```

Annexe N : Commands to set up the joystick and open gazebo

To find the name of the joystick :

```
ls -l /dev/input/
```

To get the data from the joystick :

```
roslaunch joy joy_node _dev:=/dev/input/js1 joy:=/joy1 __name:=joystick_one
```

To open the gazebo simulation :

```
roslaunch ur_gazebo ur5e_bringup.launch
```

To launch MoveIt :

```
roslaunch ur30_moveit_config moveit_planning_execution.launch sim:=true
```

Annexe O : IK joystick publisher

```
#!/usr/bin/env python3
```

```
import rospy
from sensor_msgs.msg import Joy

class ContinuousJoyRemapper:
    def __init__(self):
        # Initialiser le noeud ROS
        rospy.init_node('IK_joysticks_publisher', anonymous=True)

        self.joy_publisher = rospy.Publisher('/IK_joy_publisher', Joy, queue_size=10)

        # Le subscriber sur le topic /joy1 de votre joystick
        rospy.Subscriber('/joy1', Joy, self.joy_callback)

        # Variables pour stocker les données du joystick
        self.joy_axes = [0.0] * 8
        self.joy_buttons = [0] * 11

        rospy.loginfo("Remapper continu a démarré, s'abonne à /joy1 et publie sur /IK_joy_publisher.")

    def joy_callback(self, joy_msg):
        """
        Met à jour les variables d'instance avec les données brutes du joystick.
        """
        # Mise à jour des données du joystick lorsque le callback est appelé
        self.joy_axes = joy_msg.axes
        self.joy_buttons = joy_msg.buttons

    def run(self):
        # Définir la fréquence de publication (par exemple, 50 Hz)
        rate = rospy.Rate(250)

        while not rospy.is_shutdown():
            # Créer un nouveau message Joy pour les données mappées
            mapped_joy_msg = Joy()
            mapped_joy_msg.header.stamp = rospy.Time.now()

            # Appliquer le facteur de vitesse
            velocity_factor = 0.75

            # --- Mappage des axes ---
            # Vos règles de mappage sont appliquées ici
            # Remarque : Les indices des axes et des boutons dépendent de votre joystick.
            # Les axes et boutons sont réinitialisés à zéro pour les valeurs non mappées.

            # Initialiser les tableaux avec des valeurs par défaut
            mapped_axes = [0.0] * 6
            mapped_buttons = [0] * 6

            # Exemple de mappage basé sur votre logique :
            # Vous devrez ajuster les indices des axes et des boutons si nécessaire

            # Mappage de la translation X/Y/Z
            mapped_axes[0] = self.joy_axes[0] * velocity_factor # X
            mapped_axes[1] = (self.joy_axes[1]-0.56) * velocity_factor # Y
```

```
# Mappage de la rotation Roll/Pitch/Yaw
mapped_axes[3] = self.joy_axes[4] * velocity_factor # Roll
mapped_axes[4] = self.joy_axes[5] * velocity_factor # Pitch

# Gérer la translation Z avec les boutons
if self.joy_buttons[0] == 1: # Bouton 0
    mapped_axes[2] = velocity_factor
if self.joy_buttons[1] == 1: # Bouton 1
    mapped_axes[2] = -velocity_factor

# Gérer la rotation Yaw avec les boutons
if self.joy_buttons[2] == 1: # Bouton 2
    mapped_axes[5] = velocity_factor
if self.joy_buttons[3] == 1: # Bouton 3
    mapped_axes[5] = -velocity_factor

# Appliquer la "zone morte" pour l'axe Y (si la valeur est trop petite, elle est mise à zéro)
if abs(mapped_axes[1]) < 0.2 * velocity_factor:
    mapped_axes[1] = 0.0

# Attribuer les valeurs mappées au message
mapped_joy_msg.axes = mapped_axes
mapped_joy_msg.buttons = mapped_buttons

# Publier le message mappé
self.joy_publisher.publish(mapped_joy_msg)

# Attendre jusqu'à la prochaine itération
rate.sleep()

if __name__ == '__main__':
    try:
        remapper = ContinuousJoyRemapper()
        remapper.run()
    except rospy.ROSInterruptException:
        pass
```

Annexe P : IK control

```
#!/usr/bin/env python3
import rospy
import PyKDL as kdl
from sensor_msgs.msg import Joy, JointState
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
from urdf_parser_py.urdf import URDF
from kdl_parser_py.urdf import treeFromUrdfModel
import numpy as np

# Globals
kdl_chain = None
jac_solver = None
last_joint_state = None      # stocke le dernier JointState reçu
target_positions = None     # cible intégrée (gelée quand joystick neutre)
v_cartesian = np.zeros(6)   # [vx, vy, vz, wx, wy, wz]
gain_linear = 0.25
gain_angular = 0.25

JOINT_NAMES = [
    'shoulder_pan_joint',
    'shoulder_lift_joint',
    'elbow_joint',
    'wrist_1_joint',
    'wrist_2_joint',
    'wrist_3_joint'
]

def joy_callback(msg):
    global v_cartesian
    # adapte axes selon ton joystick si nécessaire
    v_cartesian = np.array([
        msg.axes[0] * gain_linear,
        msg.axes[1] * gain_linear,
        msg.axes[2] * gain_linear*0.5,
        msg.axes[3] * gain_angular,
        msg.axes[4] * gain_angular,
        msg.axes[5] * gain_angular
    ])

def joint_state_callback(msg):
    global last_joint_state
    last_joint_state = msg
    rospy.loginfo_throttle(2, "JointState reçu: %s", msg.position)

def main():
    global kdl_chain, jac_solver, last_joint_state, target_positions, v_cartesian
    rospy.init_node('ur5e_joystick_controller')

    # robot_description
    robot_description = rospy.get_param('robot_description', None)
    if robot_description is None:
        rospy.logerr("Paramètre 'robot_description' introuvable.")
        return

    robot = URDF.from_xml_string(robot_description)
    ok, tree = treeFromUrdfModel(robot)
    if not ok:
```

```

    rospy.logerr("Impossible de créer le KDL Tree depuis l'URDF.")
    return

try:
    kdl_chain = tree.getChain('base_link', 'tool0') # adapter si besoin
except Exception as e:
    rospy.logerr("Erreur extraction KDL chain: %s", e)
    return

if kdl_chain.getNrOfJoints() == 0:
    rospy.logerr("La chaîne KDL contient 0 articulations.")
    return

jac_solver = kdl.ChainJntToJacSolver(kdl_chain)
rospy.loginfo("KDL Chain initialized: %d joints", kdl_chain.getNrOfJoints())

rospy.Subscriber('/IK_joy_publisher', Joy, joy_callback)
rospy.Subscriber('/joint_states', JointState, joint_state_callback)
# pub = rospy.Publisher('/eff_joint_traj_controller/command', JointTrajectory, queue_size=1) # Dans gazebo
pub = rospy.Publisher('/scaled_pos_joint_traj_controller/command', JointTrajectory, queue_size=1) # ur5e

rate_hz = 100
rate = rospy.Rate(rate_hz)
delta_t = 1.0 / float(rate_hz) # integration timestep cohérent avec la boucle

vel_threshold = 1e-6 # seuil pour considérer q_dot = 0
resync_threshold = 0.5 # si target diverge trop de la mesure, resync (rad)

while not rospy.is_shutdown():
    try:
        if last_joint_state is None:
            rate.sleep()
            continue

        n_joints = kdl_chain.getNrOfJoints()
        # construire positions_ordered dans l'ordre JOINT_NAMES à partir de last_joint_state
        positions_ordered = np.zeros(n_joints)
        name_to_idx = {name: idx for idx, name in enumerate(last_joint_state.name)}
        for i, jn in enumerate(JOINT_NAMES):
            if jn in name_to_idx:
                positions_ordered[i] = last_joint_state.position[name_to_idx[jn]]
            else:
                rospy.logwarn_throttle(5, "Joint '%s' absent de /joint_states ; utilisation de 0.", jn)
                positions_ordered[i] = 0.0

        # initialisation de target_positions si première fois
        if target_positions is None:
            target_positions = positions_ordered.copy()

        # calcul de la Jacobienne (au point mesuré)
        joint_positions_kdl = kdl.JntArray(n_joints)
        for i, pos in enumerate(positions_ordered):
            joint_positions_kdl[i] = pos

        jacobian_matrix = kdl.Jacobian(n_joints)
        jac_solver.JntToJac(joint_positions_kdl, jacobian_matrix)

```

```

rows, cols = jacobian_matrix.rows(), jacobian_matrix.columns()
jac_np = np.zeros((rows, cols))
for i in range(rows):
    for j in range(cols):
        jac_np[i, j] = jacobian_matrix[i, j]

if jac_np.size == 0:
    rospy.logwarn("Jacobian vide -> pas de calcul q_dot.")
    rate.sleep()
    continue

# pseudo-inverse
q_dot_full = np.linalg.pinv(jac_np).dot(v_cartesian)

# s'assurer de la taille (on prend les premières n joints si nécessaire)
q_dot_ordered = np.zeros(len(JOINT_NAMES))
L = min(len(q_dot_full), len(q_dot_ordered))
q_dot_ordered[:L] = q_dot_full[:L]

# DEBUG
rospy.logdebug("q_dot: %s", q_dot_ordered.tolist())

# Si au moins une composante significative -> intégration, sinon freeze target_positions
if np.any(np.abs(q_dot_ordered) > vel_threshold):
    target_positions = target_positions + q_dot_ordered * delta_t

# Sécurité : si target diverge trop de la position mesurée, resynchroniser
if np.max(np.abs(target_positions - positions_ordered)) > resync_threshold:
    rospy.logwarn("Target diverge de la mesure (>%0.2f rad). Resynchronisation.", resync_threshold)
    target_positions = positions_ordered.copy()

# Publication (positions + velocities + accelerations de même taille)
jt_msg = JointTrajectory()
jt_msg.joint_names = JOINT_NAMES
point = JointTrajectoryPoint()
point.positions = target_positions.tolist()
point.velocities = q_dot_ordered.tolist()
point.accelerations = [0.0] * len(JOINT_NAMES)
point.time_from_start = rospy.Duration(delta_t)
jt_msg.points.append(point)
pub.publish(jt_msg)

except Exception as e:
    rospy.logerr("Erreur dans la boucle de contrôle : %s", e)

rate.sleep()

if __name__ == '__main__':
    try:
        main()
    except rospy.ROSInterruptException:
        pass

```